

INSTITUTO POLITÉCNICO NACIONAL

CENTRO DE INVESTIGACIÓN EN COMPUTACIÓN

SESSION-BASED RECOMMENDATION USING GRAPH NEURAL
NETWORKS AND ATTENTION MECHANISMS

T E S I S

QUE PARA OBTENER EL TÍTULO DE:

MAESTRÍA EN CIENCIAS DE LA COMPUTACIÓN

PRESENTA:

ING. ANGEL EMANUEL RODRÍGUEZ ROMÁN

ASESORES:

DR. LUIS PASTOR SÁNCHEZ FERNÁNDEZ

DR. RICARDO MENCHACA MÉNDEZ



Estados Unidos Mexicanos

Ciudad de México

2026

ABSTRACT

La recomendación basada en sesiones (session-based recommendation) es una tarea crítica en sistemas recomendadores actuales, particularmente en escenarios donde no se dispone de un perfil de usuario, es decir, en donde los usuarios interactúan en plataformas (e-commerce, redes sociales, plataformas de streaming, etc) de manera anónima. Este trabajo aborda el problema de recomendación basada en sesiones, el cual se basa en predecir el siguiente item más probable con el que el usuario interactuará dada una secuencia de interacciones ordenadas cronológicamente por marcas de tiempo. Enfoques tradicionales como el filtrado colaborativo, la factorización de matrices, entre otros, han abordado el problema de recomendación basada en sesiones, sin embargo, estos enfoques no se ajustan de manera efectiva, en particular debido a que usualmente asumen la existencia de un perfil de usuario y no están diseñados para trabajar con información secuencial. De manera similar, los enfoques basados en modelos ocultos de Markov asumen de manera estricta que el siguiente estado de la sesión depende únicamente del estado actual, ignorando prácticamente la información histórica de la sesión, la cual puede potencialmente contener información relevante para la predicción del siguiente elemento. Por otra parte, los modelos basados en cadenas de Markov no son escalables, ya que el espacio de estados puede crecer de manera importante cuando para cada estado se consideran todos los posibles siguientes estados.

Las arquitecturas basadas en Deep Learning, especialmente aquellas con capacidad para modelar información secuencial, como las redes neuronales recurrentes (RNNs), las redes tipo Long-Short-Term Memory (LSTMs) y Gated Recurrent Units (GRUs), han mostrado mejoras importantes sobre métodos tradicionales para la recomendación basada en sesiones. Más recientemente, las redes neuronales sobre grafos (GNNs) han surgido como una alternativa poderosa en esta tarea. Las GNNs, de manera natural, procesan datos que pueden ser modelados como grafos, y la tarea de recomendación basada en sesiones contiene secuencias de eventos, las cuales pueden ser modeladas como grafos. La habilidad inherente de las GNNs para capturar relaciones de alto nivel y dependencias contextuales directamente de la estructura topológica de los grafos, hacen que estas redes sean excepcionalmente buenas para modelar y encontrar patrones complejos en las sesiones.

En este trabajo, proponemos una arquitectura basada en redes neuronales sobre grafos, la cual integra dos tipos de información provenientes de las sesiones. La información secuencial y la no secuencial. De manera específica, la información secuencial es modelada mediante un

grafo dirigido, capturando conexiones valiosas y tomando en cuenta el orden cronológico de los eventos en la sesión. Mientras que la información no secuencial es modelada mediante un grafo no dirigido, intentando capturar coocurrencias entre ítems e información valiosa sobre las conexiones estructurales del grafo de sesión. Un mecanismo de atención basado en la operación de producto punto es utilizado, el cual mide la relevancia de cada uno de los ítems en la sesión. Adicionalmente, la información temporal (derivada de las marcas de tiempo en los ítems) es utilizada mediante el cómputo de un mecanismo de compuerta, bajo la premisa de que ítems con mayor tiempo de permanencia son mas relevantes e influyentes para la predicción del siguiente ítem.

En la parte experimental, se consideraron 3 datasets representativos del estado del arte: Yoochooose, Diginetica y Nowplaying. Resultados empíricos muestran que el sistema propuesto supera de manera significativa a métodos base propuestos a lo largo del tiempo, alcanzando un desempeño competente comparado con trabajos actuales representativos del estado del arte.

ABSTRACT

Session-based recommendation is a critical task in modern recommender systems, particularly in scenarios where users interact anonymously and persistent user profiles are often unavailable. This work addresses the challenge of session-based recommendation, which involves predicting the next item a user is most likely to interact with given a sequence of timestamped interactions (a session). Traditional approaches, such as collaborative filtering and matrix factorization, are not well-suited for this problem due to their reliance on long-term historical profiles. Similarly, Markov models are limited by their strict assumption of short-term dependencies, ignoring almost all past information, relying often only on the last interaction of the sequence, as well as their computational inefficiency across large state spaces.

Recent advances in deep learning have introduced highly effective paradigms for session-based modeling. While Recurrent Neural Networks (RNNs), including Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs), have demonstrated significant improvements over traditional shallow methods, Graph Neural Networks (GNNs) have recently emerged as a powerful alternative. GNNs naturally process session data as graphs. These networks have an innate ability to capture high-order relationships coming from the topological structure of the graphs, which makes them a powerful framework to capture complex patterns present in sessions.

In this work, we propose an architecture based on GNNs that integrates both sequential and non-sequential session information. Specifically, a directed graph representation is utilized to capture the strict chronological order of interactions, while an undirected graph models item co-occurrences and structural connectivity, completely independent of the order of the interactions. The model is further enhanced by a dot-product attention mechanism that dynamically weights the relevance of individual items within the session. Additionally, time dynamics (derived from item timestamps) are modeled using a temporal gate that balances the importance of semantic features (item embeddings) against temporal signals. This gated temporal information is subsequently injected into the computation of the attention scores via element-wise multiplication.

Experimental evaluations were conducted using three benchmark datasets for session-based recommendation: *Yoochoose*, *Diginetica*, and *Nowplaying*. The empirical results demonstrate that the proposed framework significantly outperforms traditional baselines and achieves highly competitive, state-of-the-art performance when compared against some recent representative

methods.

ACKNOWLEDGEMENTS

I thank my children, Emily and Daniel, my life partner Diana, my parents Raul and Hilda and my brother Jesus. I thank my professors who guide me through this journey and I deeply admire and respect. I also thank SECIHTI for the financial support.

ÍNDICE

	Pág.
Abstract	iii
Abstract	v
Índice	ix
List of figures	xi
List of tables	xiii
1 Introduction	1
1.1 Motivation	3
1.2 Problem Statement	3
1.3 Objectives	3
1.3.1 Main objective	3
1.3.2 Specific objectives	4
1.4 Research Questions	4
2 Fundamentals	5
2.1 Recurrent Neural Networks (RNN)	5
2.1.1 Long Short-Term Memory (LSTM) Networks	5
2.1.2 Gated Recurrent Unit (GRU)	6
2.2 Graphs	7
2.3 The Graph Neural Network Framework	8
2.3.1 Defining a Graph Neural Network	8
2.3.2 Permutation Invariance Property	9
2.3.3 Permutation Equivariance Property	10
2.3.4 A Basic GNN	10
2.3.5 Graph Convolutional Networks (GCN)	11
2.3.6 Gated Graph Neural Networks (GGNN)	12
2.3.7 Popular architectures based on graphs	13

3	Related Work	15
3.1	Collaborative Filtering	15
3.2	Markov Decision Processes	16
3.3	Factor Models and K-Nearest Neighbors (KNN) Approaches	16
3.4	Recurrent Neural Network (RNN) Approaches	16
3.5	Graph Neural Network Approaches	17
4	Solution Approach	21
4.1	Notation	21
4.2	Proposed architecture overview	23
4.3	Non-sequential Modeling	23
4.4	Sequential Modeling	26
4.5	Attention Layer	30
4.6	Prediction Layer	32
4.6.1	Model Evaluation	32
4.6.2	Loss function	33
5	Experimental Results	35
5.1	Data	35
5.2	Time analysis	36
5.3	Experimental setup	37
5.4	Results obtained	38
5.5	Comparison with some baselines	40
5.6	Comparisson with some recent methods	43
6	Conclusions and future work	45
	References	49

LIST OF FIGURES

FIGURE	Pág.
4.1 High-level diagram of the proposed architecture	23
4.2 A toy example of a session containing 8 interactions with 5 unique items: from item v_{s1} to item v_{s5} . The respective undirected session graph is constructed, indicating connections between items in the session. And the respective symmetric adjacency matrix $\mathbf{A}$ is presented.	24
4.3 Thumbnail view of the non-sequential layer	25
4.4 Example of sequential modeling, the example illustrates an anonymous session and their respective session graphs. The graphs model incoming and outgoing connections with their respective adjacency matrices $\mathbf{A}_{in}$ and $\mathbf{A}_{out}$	27
4.5 Matrices $\mathbf{A}_{in}$ (left) and $\mathbf{A}_{out}$ (right) and their respective normalized matrices $\tilde{\mathbf{A}}_{in}$ and $\tilde{\mathbf{A}}_{out}$	28
4.6 Thumbnail view of the sequential layer	29
4.7 Thumbnail view of the attention layer	30
4.8 Thumbnail view of the prediction layer	32
5.1 Analysis of the temporal information on the three datasets. For every chart, the horizontal axis indicates a discrete bin (obtained from the logarithm of the dwell times computed between the items in all sessions from the dataset), and for the charts in the top, the vertical axis, represents the frequency for each bin computed across the entire dataset. For charts in the bottom, the vertical axis is the frequency of each bin using a logarithmic scale too. The pictures are placed from left to right, where Yoochoose is the dataset containing the smallest dwell times and Nowplaying containing the largest dwell times. Since this last dataset comes from the music domain, it contains larger dwell times between items since songs are usually larger events than interactions the e-commerce domain.	37

LIST OF TABLES

TABLE	Pág.
2.1 Key Graph Neural Network Architectures	13
4.1 Notation table	22
5.1 Statistics of Yoochoose 1/64, Diginetica, and Nowplaying Datasets after preprocessing	36
5.2 Combined results for Diginetica, Yoochoose, and Nowplaying datasets	38
5.3 Dataset Statistics for Short and Long Sequences	39
5.4 Short and Long Sessions	39
5.5 Dataset Evaluation Performance Metrics	40
5.6 Performance Comparison Across Different Baselines and Datasets	42
5.7 Performance Comparison with some Recent Methods (2020–2025)	43

INTRODUCTION

Recommender Systems (RS) aim to predict user preferences and suggest relevant items (e.g., products, media, services) by leveraging historical interactions. By deeply analyzing user behavior, RS significantly enhance the user experience across a wide range of domains, including social networks, streaming platforms, e-commerce, among others.

Session-based recommendation (SB-recommendation) is an important subfield with multiple potential applications that has recently gained substantial attention within the recommender systems community. In this specific setting, the objective is to predict the next item a user will interact with, given an ongoing sequence of actions (e.g., predicting the next click in a browsing session) [14, 20, 22]. Modern e-commerce, news, and media platforms frequently cannot reliably track user identities over time, often due to strict privacy regulations or the technical limitations of cookies and browser fingerprinting. Thus, each session should be considered independently as an anonymous sequence of interactions. The process of recommendation must not rely on a persistent user profile but only on the behavioral patterns contained in the sequence itself. [20].

Over the years, the task of session-based recommendation has been tackled under several approaches. Earlier methods used to adapt Collaborative Filtering (CF) to address the task, adapting techniques such as Matrix Factorization [1, 2, 10, 29, 30] to this problem. These traditional approaches usually model the tasks as the completion of a sparse matrix, indicating connections between users and items. Despite the fact that these methods have been adapted for SB-recommendation, they are not intended to model sequential data, since they were originally designed to address traditional recommendation scenarios, where information about users and items exists, and a user profile is often available.

Other traditional approaches model the problem using Markov Chains [11, 12]. However, these models typically rely on a strong first-order independence assumption, meaning they predict the next item based solely on the very last interaction, ignoring the rich context of earlier

actions within the session that could heavily influence the user’s final intent. While it is possible to build higher-order Markov models to look further back into the sequence, the state space grows exponentially, making them computationally impractical for large catalogs. Similarly, neighborhood-based methods (such as K-Nearest Neighbors) [13] have been used to tackle the problem based on item-to-item co-occurrence. While effective in certain scenarios, these methods inherently lack the capacity to model sequences, treating the session merely as a collection of similarities rather than a chronologically ordered journey.

To overcome these structural limitations, Deep Learning has taken center stage in this area. Initial approaches using traditional feed-forward neural networks (MLPs) [15, 16] achieved reasonable results, but they natively struggle with the variable-length nature of user sessions. Consequently, Recurrent Neural Networks (RNNs) [14, 20, 22, 32] were adapted for the task. Because they are explicitly designed to model sequential data via mechanisms like Gated Recurrent Units (GRUs) and Long Short-Term Memory (LSTM) networks. RNN-based models have shown highly competitive results, frequently surpassing traditional shallow baselines. On the same page, modern architectures leveraging pure attention mechanisms and Transformer-based models have also been widely implemented, capturing complex long-range dependencies and competing directly with the best RNN approaches.

Most recently, Graph Neural Networks (GNNs) [17, 19, 21, 23, 26, 31] have emerged as a highly sophisticated and powerful framework for this problem. By modeling individual sessions as graphs, GNNs can look beyond strict left-to-right sequential ordering to capture complex item transitions and structural patterns through neighborhood aggregation. Frequently combined with attention mechanisms to construct the final session representation, GNNs have achieved significant results and have firmly established themselves as the current state-of-the-art for session-based recommendation.

In this work, we propose a GNN-based architecture that incorporates both sequential and non-sequential information from sessions. Here, a directed graph representation of the session is used to capture the chronological order of interactions, while an undirected graph representation captures item co-occurrence and indicates connections between items in the session (Without taking the order of the events into account). The model is further enhanced with an attention mechanism (dot product attention) that dynamically weighs the importance of items in the session. Temporal information (item timestamps) is modeled using a temporal gate that measures the importance between semantic (information from item embeddings) and temporal information. And is included during the computation of the attention scores via element-wise multiplication.

Experimental evaluations are conducted using representative datasets for session-based recommendation (Yoochoose, Diginetica and Nowplaying). Preliminary results suggest that the proposed framework exceeds representative baselines and achieves state-of-the-art performance, showing competitive results against recent methods.

1.1 Motivation

Recommender Systems (RS) leverage machine learning algorithms to predict user interests, helping individuals discover relevant items while simultaneously guiding the user journey, driving platform traffic, customer satisfaction, engagement, and ultimately, sales. In recent years, due to the exponential growth of data on the internet, RS have become an indispensable component of digital platforms ranging from e-commerce websites to entertainment applications, serving as the primary mechanism to alleviate information overload.

A well-known challenge within traditional recommendation frameworks is the cold-start problem [4]. This occurs when a system lacks sufficient historical data to accurately model user preferences, making it extremely difficult to generate personalized recommendations when a new user first interacts with the platform. However, in session-based recommendation, this limitation is naturally mitigated. Because session-based models do not rely on persistent user profiles, they generate predictions relying strictly on the sequence of ongoing interactions, which are frequently treated as anonymous sessions.

To tackle the complexities of these sequential behaviors, modern recommendation systems have shifted toward machine learning solutions. Current state-of-the-art approaches implement and combine advanced deep learning architectures—such as Recurrent Neural Networks (RNNs), attention mechanisms, transformer-based models, and Graph Neural Networks to successfully capture both short-term transitions and complex, high-order item dependencies.

1.2 Problem Statement

The primary problem addressed in this work is predicting the next item to be visited or interacted with, given an anonymous sequence of historical interactions. Formally, let V denote the set of all unique items and S denote the set of all sessions within the dataset. A specific session $s \in S$ is defined as a sequence of items from V , strictly ordered by their chronological timestamps, such that $s = [v_{s,1}, v_{s,2}, \dots, v_{s,n-1}]$, where each element $v_{s,i} \in V$. The core objective of the session-based recommendation task is to accurately predict the next target item $v_{s,n} \in V$ that naturally follows the observed sequence s .

1.3 Objectives

1.3.1 Main objective

The objective of this work is to develop a session-based recommendation system using Graph Neural Networks and Attention Mechanisms, specifically evaluating the impact of sequential and non-sequential information, as well as temporal dynamics, on the overall recommendation performance.

1.3.2 Specific objectives

- To conduct a review of similar works for sequential recommendation, focusing specifically on approaches leveraging Graph Neural Networks (GNNs) and attention mechanisms, to synthesize key limitations and opportunities in the current research landscape.
- To propose and develop a recommendation architecture, integrating graph neural networks to obtain embedding representations of the nodes from session graphs.
- To incorporate and evaluate the effect of sequential and non-sequential information from the session graph within the proposed GNN framework, and analyze its impact on the final recommendation performance.
- To integrate an attention layer to obtain the final session representation based on attention weights computed for each item present in the session.
- To model temporal dynamics by incorporating time information (e.g., normalized time elapsed between sequential interactions), capturing the importance of some items according to their time span in the attention layer.
- To evaluate the proposed architecture against similar session-based recommendation baselines using standard benchmark datasets, analyzing results obtained in performance metrics, specifically Precision@K and MRR@K.
- To perform ablation studies to validate the contribution of each component: sequential, non-sequential information, and time dynamics.

1.4 Research Questions

We aim to explore the following research questions (RQ):

- RQ1: What is the effect of sequential and non-sequential information on the recommendation performance?
- RQ2: What is the effect of temporal dynamics in the final recommendation? Does the effect depend of the nature of the temporal information present in the data?
- RQ3: What is the performance of the model for short and long sessions?
- RQ4: What is the performance of the model for different top-k recommendations?
- RQ5: Does the proposed method surpass traditional baselines in session-based recommendation?
- RQ6: Can the proposed architecture achieve state-of-the-art results?

FUNDAMENTALS

2.1 Recurrent Neural Networks (RNN)

Recurrent neural networks are a special set of neural network architectures designed to model and detect patterns in sequential data. Such data can be handwriting, genomes, text or numerical time series which are often produced in industry settings (e.g. stock markets or sensors) [37]. One main difference between RNNs and Multi-Layer Perceptrons (MLPs), also known as Feedforward Networks, is the way information flows through the network. While MLPs pass information through the network without cycles, RNNs have backward cycles, allowing the information to be transmitted back again into the network, making the next iteration be aware of the previous context known as hidden state. The simple RNN is formally defined as follows [36]:

$$\mathbf{h}_t = g(\mathbf{W}[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}) \quad (2.1)$$

Where $\mathbf{h}_t$ is the hidden state, $\mathbf{x}_t$ is the input vector at time t (the current input), the term $\mathbf{W}[\mathbf{h}_{t-1}; \mathbf{x}_t]$ represents the projection for $\mathbf{x}_t$ similar to the MLP operation, but introducing the concatenation of the current input with the previous hidden state (acting as a feedback loop), g is an activation function such as the sigmoid function, the hyperbolic tangent function, or the rectified Linear Unit (ReLU). $\mathbf{W}$, $\mathbf{U}$ and $\mathbf{b}$ are learnable parameters for linear projections.

2.1.1 Long Short-Term Memory (LSTM) Networks

While traditional Recurrent Neural Networks (RNNs) are theoretically capable of handling sequential data, showing competitive results [37], they severely struggle to learn long-term sequences in practice due to the vanishing and exploding gradient scenarios. To address these constraints, Hochreiter and Schmidhuber presented a variant of a RNN, the Long Short-Term

Memory (LSTM) network [22]. The core innovation of the LSTM architecture is the introduction of a central memory component, known as the cell state, regulated by a series of differentiable gating mechanisms that control the flow of information across time steps.

The cell state $\mathbf{c}_t$ runs straight down the entire chain of the sequence, acting as a conveyor belt that allows gradients to flow virtually unchanged. The network can strictly add or remove information from this cell state using three distinct gates: the forget gate, the input gate, and the output gate. At a given time step t , the LSTM unit takes the current input vector $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$ to compute the network updates.

First, the forget gate $\mathbf{f}_t$ determines what information should be discarded from the previous cell state $\mathbf{c}_{t-1}$. It applies a sigmoid function (σ) to output a value between 0 and 1 for each value in the cell state, where 1 represents "keep all the information" and 0 represents "discard everything":

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f) \quad (2.2)$$

Next, the network determines what new information will be stored in the cell state in a two-part process. The input gate $\mathbf{i}_t$ decides which values will be updated, while a tanh layer generates a vector of new candidate values $\tilde{\mathbf{c}}_t$ that could potentially be added to the state:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_i) \quad (2.3)$$

$$\tilde{\mathbf{c}}_t = g(\mathbf{W}_c[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_c) \quad (2.4)$$

The old cell state $\mathbf{c}_{t-1}$ is then updated to the new cell state $\mathbf{c}_t$. This is achieved by multiplying the old state by the forget gate $\mathbf{f}_t$, and adding the scaled candidate values:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (2.5)$$

where $\odot$ denotes the element-wise (Hadamard) product. Finally, the network computes the output hidden state $\mathbf{h}_t$, which is a filtered version of the newly updated cell state. The output gate $\mathbf{o}_t$ dictates what parts of the cell state will be output, and the cell state is passed through a tanh function to push the values between -1 and 1 before being multiplied by the output gate:

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_o) \quad (2.6)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot g(\mathbf{c}_t) \quad (2.7)$$

Here, $\mathbf{W}_f, \mathbf{W}_i, \mathbf{W}_c, \mathbf{W}_o$ represent the trainable weight matrices, and $\mathbf{b}_f, \mathbf{b}_i, \mathbf{b}_c, \mathbf{b}_o$ are the corresponding bias vectors. Through this sophisticated gating mechanism, LSTMs effectively capture both short-term sequential transitions and complex, long-range dependencies within the data.

2.1.2 Gated Recurrent Unit (GRU)

The Gated Recurrent Unit (GRU) is a RNN variant that reduces the gating signals to just two from the LSTM RNN Model. The 2 gates are called the update gate $\mathbf{z}_t$ and the reset gate $\mathbf{r}_t$ [36].

The GRU is formally defined as follows:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_z) \quad (2.8)$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_r) \quad (2.9)$$

From these two gates, we obtain the candidate state and the final representation:

$$\mathbf{h}_t = (\mathbf{1} - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad (2.10)$$

$$\tilde{\mathbf{h}}_t = g(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h) \quad (2.11)$$

Intuitively, these equations rule the flow of information through the GRU cell by wisely preserving or discarding memory. The update gate $\mathbf{z}_t$ determines the balance between historical memory and newly generated information; a value close to 0 forces the model to retain the previous state $\mathbf{h}_{t-1}$, while a value close to 1 replaces it with the new candidate state. The reset gate $\mathbf{r}_t$ determines how much of the past memory is actually relevant for computing the next state. If $\mathbf{r}_t$ is close to 0, the model effectively drops the past and focuses solely on the current input $\mathbf{x}_t$. The candidate state $\tilde{\mathbf{h}}_t$ represents the proposed new memory content for the current time step, generated by applying a non-linear activation function (g) to the current input and the explicitly filtered past state. Finally, the **final representation** $\mathbf{h}_t$ is constructed as a linear interpolation between the previous state and the new candidate state. This additive update mechanism, directly controlled by $\mathbf{z}_t$, allows the GRU to carry information across many time steps unchanged, effectively mitigating the vanishing gradient problem while remaining computationally more efficient than an LSTM.

2.2 Graphs

We define a graph as a set that contains two sets. Nodes (V) and Edges (E):

$$G = \{V, E\} \quad (2.12)$$

V contains nodes (e.g. items in a session) and E indicates connections between them. Depending on its edges, a graph might be directed or undirected. In a directed graph, the edges point in a specific direction. An edge from node u to node v is distinct from an edge from node v to node u . Therefore, the edge set E_d consists of ordered pairs of vertices.

$$G_d = \{V, E_d\} \quad (2.13)$$

Where the edge set is defined as a subset of the Cartesian product of V with itself:

$$E_d \subseteq \{(u, v) \mid u, v \in V\} \quad (2.14)$$

Where, (u, v) represents a directed edge that points from node u to node v . $(u, v) \neq (v, u) \forall u \neq v$. An undirected graph, contains symmetrical connections. If node u is connected to node v , then

node v is simultaneously connected to node u . Therefore, the edge set E_u consists of unordered pairs (sets of two distinct elements).

$$G_u = \{V, E_u\} \quad (2.15)$$

The edge set E_u is defined as:

$$E_u \subseteq \{\{u, v\} \mid u, v \in V \text{ and } u \neq v\} \quad (2.16)$$

In practice, graphs can be represented as matrices or adjacency lists. In this work, graphs are represented as matrices. Since we used the framework Pytorch, we take advantage of Graphical Processing Units (GPUs) to perform matrix operations efficiently. Figure [a] shows a representation of a session graph as an adjacency matrix.

2.3 The Graph Neural Network Framework

The graph neural network formalism is a general framework for running deep neural networks on graphs, aiming to build node representations that are deeply informed by the graph's structure. As well as any feature information present in the data. The main challenge is to develop encoders for graph structure data. Since the traditional deep learning toolbox does not work on it. Thus, a new kind of deep learning architecture needs to be defined [27].

As mentioned earlier, Graph Neural Networks (GNNs) aim to generate representations of nodes. These representations are embeddings of the original nodes. Hence, an embedding space is generated by GNNs, where the main idea is that nodes that are close in the embedding space (using some distance metric such as the dot product) are also close in the original graph.

2.3.1 Defining a Graph Neural Network

A GNN consists of two key components. The aggregate and update functions. Aggregate is defined as follows:

$$\mathbf{m}_{N(u)}^{(k)} = \text{AGGREGATE}(\{\mathbf{h}_v^{(k)}, \forall v \in N(u)\}) \quad (2.17)$$

As seen in equation 2.17, a set of neighbor feature vectors is taken, $\{\mathbf{h}_v^{(k)}, \forall v \in N(u)\}$, and then, combined into a single one, fixed-size vector. This operation acts as a compressed and summarized representation of the node's entire neighborhood. Where $\mathbf{m}_{N(u)}^{(k)}$ represents the aggregated "message" from the neighborhood of node u at layer k .

The update function is defined as follows:

$$\mathbf{h}_u^{(k+1)} = \text{UPDATE}^{(k)}(\mathbf{h}_u^{(k)}, \mathbf{m}_{N(u)}^{(k)}) \quad \mathbf{h}_u^{(k+1)} = \text{UPDATE}^{(k)}(\mathbf{h}_u^{(k)}, \mathbf{m}_{N(u)}^{(k)}) \quad (2.18)$$

The *UPDATE* function is typically a linear transformation followed by a non-linearity (an activation function such as ReLU), typically a neural network with learnable weights. But

could be any other arbitrary differentiable function. *UPDATE* provides the embedding's new vector representation $\mathbf{h}_u^{(k+1)}$. It takes the result from the *AGGREGATE* function as well as the embedding representation of the previous time $\mathbf{h}_u^{(k)}$ as input. Note that if we only used the aggregated neighborhood message, the node would lose its original information, since the update would rely only on the information from the node's neighborhood, losing the node's identity. By using both, the node representation and its message, the GNN allows the node to both retain its own identity and incorporate information from its local graph structure. At the initial step ($k = 0$), the node embeddings are initialized using their corresponding input features. That is: $\mathbf{h}_u^{(0)} = \mathbf{x}_u, \forall u \in V$. The final embedding representation for each node is obtained after running K iterations (e.g K layers) of the GNN message passing. The general formula for a single layer in a message passing GNN is defined as follows:

$$\mathbf{h}_u^{(k+1)} = \text{UPDATE}^k \left(\mathbf{h}_u^{(k)}, \text{AGGREGATE} \left(\{\mathbf{h}_v^{(k)}, \forall v \in N(u)\} \right) \right) \quad (2.19)$$

For a node u , its node embedding $\mathbf{h}_u$ is transformed as it passes through layer k to produce its output at layer $k + 1$. Where:

- $\mathbf{h}_u^{(k)}$ is the feature vector (or hidden state) of node u at the layer k
- $N(u)$ is the set of all neighbors of node u
- The *AGGREGATE* function (such as min, max or sum) compresses the feature vectors of all neighbors $\{\mathbf{h}_v^{(k)}\}$ into a single vector.
- The *UPDATE* function, combines the node's own vector $\mathbf{h}_u^{(k)}$ with the aggregated neighborhood vector.
- $\mathbf{h}_u^{(k+1)}$ is the final output: the new feature vector for node u at the next layer, $k + 1$

2.3.2 Permutation Invariance Property

Because the neighbors of a node lack a natural or predefined ordering, any neighborhood aggregation function must be permutation invariant. More formally, let $G = (V, E)$ be a graph represented by an adjacency matrix $A \in \mathbb{R}^{|V| \times |V|}$, and let $X \in \mathbb{R}^{|V| \times d}$ denote the node feature matrix, where d is the input feature dimension. A graph-level function $f : \mathbb{R}^{|V| \times |V|} \times \mathbb{R}^{|V| \times d} \rightarrow \mathbb{R}^d$ that maps the graph structure and node features to a d -dimensional latent representation is considered permutation invariant if it satisfies:

$$f(A, X) = f(PAP^T, PX) \quad (2.20)$$

where $P \in \{0, 1\}^{|V| \times |V|}$ is an arbitrary permutation matrix. Essentially, permutation invariance guarantees that the model's output is independent of the arbitrary indexing of the nodes in the graph representation.

2.3.3 Permutation Equivariance Property

While graph-level representations must be invariant to node ordering, functions operating at the node level—such as those mapping input features to node-specific latent embeddings—must satisfy the property of permutation equivariance. Formally, a node-level function $f : \mathbb{R}^{|V| \times |V|} \times \mathbb{R}^{|V| \times d} \rightarrow \mathbb{R}^{|V| \times d'}$ maps the adjacency matrix A and node feature matrix X to an updated node representation of dimension d' . The function f is said to be permutation equivariant if, for any permutation matrix $P \in \{0, 1\}^{|V| \times |V|}$, it satisfies:

$$f(PAP^T, PX) = Pf(A, X) \quad (2.21)$$

In other words, applying a structural permutation to the input graph and subsequently processing it yields the exact same node embeddings as processing the original graph and then permuting the output rows. This property guarantees that the learned representation of a specific node depends strictly on its structural position and local neighborhood within the graph, rather than its arbitrary index in the data matrix.

The structure of a graph encodes a significant amount of implicit information, which is often hard to capture using traditional learning techniques. By representing data as a graph, these hidden relationships become explicitly modeled. This is where Graph Neural Networks (GNNs) come in. They are designed to take advantage of the relational structure of the data to enhance performance on tasks such as rating prediction or similar content retrieval, which are common in recommender systems.

The primary goal of GNNs is to learn better representations (embeddings) of nodes by aggregating information from their neighbors. Although GNNs can also be applied to learn representations for edges or even entire graphs, this work focuses specifically on node-level embeddings.

GNNs update node representations through an iterative process that aggregates feature information from a node's local neighborhood. The aggregation function defines how the features from neighbors are combined. Common strategies include mean pooling, max pooling, and others. These strategies should be permutation invariant operations [5].

2.3.4 A Basic GNN

According to Hamilton [27], a simplified version of the foundational GNN architectures—originally proposed by Merkwirth and Lengauer (2005) and Scarselli et al. (2009)—can be defined by the following update rule:

$$h_u^{(k)} = \sigma \left(W^{(k)} h_u^{(k-1)} + W_{\text{neigh}}^{(k)} \sum_{v \in N(u)} h_v^{(k-1)} + b^{(k)} \right) \quad (2.22)$$

where:

- $h_u^{(k)}$ is the hidden representation (or embedding) of node u at the k -th iteration (or layer).
- $h_u^{(k-1)}$ is the representation of node u from the previous iteration.
- $N(u)$ denotes the set of neighboring nodes connected to u .
- $h_v^{(k-1)}$ represents the embeddings of the neighboring nodes v from the previous iteration.
- $W^{(k)}, W_{\text{neigh}}^{(k)} \in \mathbb{R}^{d^{(k)} \times d^{(k-1)}}$ are the trainable weight matrices responsible for transforming the node's own representation and the aggregated neighborhood representation, respectively.
- $b^{(k)} \in \mathbb{R}^{d^{(k)}}$ is the trainable bias vector at layer k .
- σ is a non-linear activation function (such as ReLU or tanh).

2.3.5 Graph Convolutional Networks (GCN)

The Graph Convolutional Network (GCN) is a widely adopted GNN architecture. It relies on a message-passing function that incorporates both a symmetric-normalized aggregation and includes self-loops to update node representations. The message-passing function is defined as follows:

$$\mathbf{h}_u^{(k)} = \sigma \left(\mathbf{W}^{(k)} \sum_{v \in N(u) \cup \{u\}} \frac{\mathbf{h}_v^{(k-1)}}{\sqrt{|N(u)||N(v)|}} \right) \quad (2.23)$$

where:

- $\mathbf{h}_u^{(k)}$ is the embedded representation of node u at the k -th iteration (or layer).
- $\mathbf{W}^{(k)}$ represents the trainable weight matrix applied at layer k .
- $N(u) \cup \{u\}$ denotes the neighborhood of node u , explicitly incorporating the node itself to ensure its own previous features are preserved during the update (the self-loop approach).
- $\mathbf{h}_v^{(k-1)}$ is the representation of node v (which can be a neighbor or node u itself) from the previous layer.
- $|N(u)|$ and $|N(v)|$ denote the degree (number of connections) of nodes u and v , respectively. The scaling factor $\frac{1}{\sqrt{|N(u)||N(v)|}}$ serves as a symmetric normalization term, preventing the feature representations from exploding for nodes with exceptionally high connectivity.
- σ is a non-linear activation function, most commonly ReLU.

2.3.6 Gated Graph Neural Networks (GGNN)

Gated Graph Neural Networks (GGNNs), originally introduced by Li et al. [25], expand the standard GNN framework by incorporating recurrent mechanisms to update node representations across propagation steps. Because they are highly effective at capturing complex transitions and long-term dependencies, GGNNs have become a foundational architecture for session-based recommendation, most notably serving as the core message-passing mechanism in models such as SR-GNN [26].

Unlike basic GCNs that use simple linear transformations to update node states, GGNNs employ a Gated Recurrent Unit (GRU) to carefully control the flow of information during the update phase. At each propagation step k , a node u first aggregates information from its local neighborhood to form a message vector $\mathbf{a}_u^{(k)}$. Because user sessions are naturally chronological, they are typically modeled as directed graphs. Therefore, the aggregation process captures both incoming and outgoing connections:

$$\mathbf{a}_u^{(k)} = \mathbf{A}_{u,in}[\mathbf{h}_1^{(k-1)}, \dots, \mathbf{h}_n^{(k-1)}]^\top \mathbf{W}_{in} + \mathbf{A}_{u,out}[\mathbf{h}_1^{(k-1)}, \dots, \mathbf{h}_n^{(k-1)}]^\top \mathbf{W}_{out} \quad (2.24)$$

Once the structural neighborhood information $\mathbf{a}_u^{(k)}$ is gathered, the GGNN updates the hidden state of node u using the GRU mechanism. This process is governed by a series of differentiable gates:

$$\mathbf{z}_u^{(k)} = \sigma(\mathbf{W}_z \mathbf{a}_u^{(k)} + \mathbf{U}_z \mathbf{h}_u^{(k-1)}) \quad (2.25)$$

$$\mathbf{r}_u^{(k)} = \sigma(\mathbf{W}_r \mathbf{a}_u^{(k)} + \mathbf{U}_r \mathbf{h}_u^{(k-1)}) \quad (2.26)$$

$$\tilde{\mathbf{h}}_u^{(k)} = \tanh(\mathbf{W}_h \mathbf{a}_u^{(k)} + \mathbf{U}_h(\mathbf{r}_u^{(k)} \odot \mathbf{h}_u^{(k-1)})) \quad (2.27)$$

$$\mathbf{h}_u^{(k)} = (1 - \mathbf{z}_u^{(k)}) \odot \mathbf{h}_u^{(k-1)} + \mathbf{z}_u^{(k)} \odot \tilde{\mathbf{h}}_u^{(k)} \quad (2.28)$$

where:

- $\mathbf{a}_u^{(k)}$ is the aggregated message vector for node u at step k , incorporating the structural information from its neighbors.
- $\mathbf{A}_{u,in}$ and $\mathbf{A}_{u,out}$ denote the specific vectors from the adjacency matrix corresponding to the incoming and outgoing edges of node u .
- $\mathbf{W}_{in}$ and $\mathbf{W}_{out}$ are trainable weight matrices that linearly transform the features of the incoming and outgoing neighbors.
- $\mathbf{z}_u^{(k)}$ and $\mathbf{r}_u^{(k)}$ represent the update and reset gates of the GRU, respectively. These gates dynamically dictate how much past information to retain and how much new neighborhood information to integrate.
- $\tilde{\mathbf{h}}_u^{(k)}$ is the newly generated candidate hidden state.

- $\mathbf{h}_u^{(k)}$ is the final updated embedded representation of node u .
- $\mathbf{W}_z, \mathbf{W}_r, \mathbf{W}_h$ and $\mathbf{U}_z, \mathbf{U}_r, \mathbf{U}_h$ are the core trainable weight matrices of the GRU mechanism.
- σ represents the sigmoid activation function, and $\odot$ denotes the element-wise (Hadamard) product.

By utilizing these sophisticated recurrent gating mechanisms, GGNNs effectively mitigate the vanishing gradient problem during spatial information propagation. This ensures that items retain their unique, underlying features even as complex sequential patterns are aggregated across the graph.

2.3.7 Popular architectures based on graphs

Table 2.1: Key Graph Neural Network Architectures

GNN Type		Mechanism
Random Walk based		Explores graph topology by simulating random paths to capture structural proximity and learn node representations [17].
Graph Convolutional Networks (GCNs)		Aggregates features from local node neighborhoods using localized, shared convolutional filters [17].
Graph Attention Networks (GATs)		Dynamically weights the specific contributions of neighboring nodes during feature aggregation using an attention mechanism [34].
GraphSAGE		Samples a fixed-size local neighborhood and applies aggregation functions (e.g., mean, pooling) to generate node embeddings inductively [35].
Recurrent GNNs (e.g., GGNNs)		Employs recurrent units (such as GRUs) to update node states iteratively, effectively capturing sequential graph patterns [17].

RELATED WORK**3.1 Collaborative Filtering**

Collaborative Filtering (CF) is a foundational algorithm widely utilized in traditional recommender systems. It leverages the known preferences of a group of users to predict the unknown preferences of target users [?]. CF techniques are generally categorized into two main paradigms: memory-based and model-based approaches.

Memory-based methods compute predictions by utilizing the entire user-item interaction matrix (or a sample), to identify groups of similar users or items (neighborhoods). A prominent example of this paradigm is the neighborhood-based CF algorithm. Conversely, model-based techniques leverage historical rating data to train predictive machine learning models, with well-known examples including Bayesian belief networks and clustering models.

Traditionally, these core CF methods are not well-suited for Session-Based (SB) recommendation. Since they inherently rely on the existence of user profiles, which are explicitly unavailable in anonymous session scenarios. However, early efforts have attempted to adapt these techniques for sequential data. For instance, in 2011, the authors of [1] proposed a CF approach for sequential music recommendation. Because explicit user profiles were not available, they structured the data into individual listening sessions with the objective of predicting the next song. To overcome the lack of user histories, the authors identified similar sessions based on shared item occurrences across the dataset, and subsequently applied a traditional CF algorithm utilizing cosine similarity to generate the final recommendations.

3.2 Markov Decision Processes

Other approaches have also shown promising results in SB-based recommendation, notably those based on Markov Chains [11]. In this formulation, the recommendation problem is modeled as a Markov Decision Process (MDP). An MDP is defined as a tuple of 4 elements: $\langle S, A, R, \delta \rangle$ [12], where S is a set containing all states, A is a set of actions, R is the reward function, and δ is the mapping function of states and actions. In particular, the problem of session-based recommendation can be modeled as the simplest MDP. This mechanism is analogous to a first-order Markov chain, in which the next recommendation is determined by the transition probability between items, and some strong independence assumptions are made: the current event in a session depends only on the immediately preceding event. While Markov-based models can yield competitive results, they face significant scalability challenges when trying to consider all possible sequences of user selections. Hence, it becomes computationally expensive, and the resulting state space often grows unmanageably large in real-world scenarios.

3.3 Factor Models and K-Nearest Neighbors (KNN) Approaches

Historically, the most prevalent methodologies in recommender systems have been factor models. However, similar to traditional Collaborative Filtering, these techniques typically rely on the existence of persistent user profiles.

Factor models [5–8] operate by decomposing a sparse user-item interaction matrix into a lower-dimensional latent space, generating a d -dimensional vector for each user and item in the dataset. The recommendation task is thus formulated as a matrix completion (or reconstruction) problem. The learned latent factor vectors are leveraged to estimate missing entries, most commonly by computing the dot product between a specific user’s and item’s latent representations. Despite their success in traditional settings, factor models are inherently challenging to apply to SB-recommendation. In these anonymous environments, explicit user profiles are frequently unavailable; more importantly, the standard factorization paradigm lacks the capability to take the chronological sequence of items within a session into account.

Conversely, neighborhood methods (such as KNN) rely on measuring similarities between items or users based on co-occurrence frequencies within historical profiles or individual sessions. Because item-to-item similarity can be calculated independently of long-term user identity, these co-occurrence approaches have been more readily adapted and extensively employed as baseline models in session-based recommendation tasks [9, 10].

3.4 Recurrent Neural Network (RNN) Approaches

Recurrent Neural Networks (RNNs) emerged as the natural next step to overcome some limitations of the previously mentioned baselines. By maintaining a continuous hidden state that acts

as a dynamic memory over the entire sequence, RNNs can naturally capture both short-term transitions and complex, long-range sequential dependencies.

Proposed in 2016, the Gated Recurrent Unit for Recommendation (GRU4REC) [20] was a pioneering deep learning architecture that established a new state-of-the-art for session-based recommendation. By adapting Recurrent Neural Networks—specifically utilizing Gated Recurrent Units (GRUs)—the model successfully captures sequential user patterns while effectively mitigating the vanishing gradient problem associated with standard RNNs. To handle scale and ensure training efficiency, the authors introduced practical novelties such as session-parallel mini-batches and popularity-based sampling. Furthermore, after rigorously evaluating pointwise, pairwise, and listwise ranking losses, they determined that pairwise ranking (specifically utilizing BPR and TOP1) delivered the most stable and superior performance by enforcing a strict positive-over-negative item ranking constraint.

Architecturally, GRU4REC models the input session state using a 1-of-N (one-hot) encoding vector representing the entire item catalog. Extensive experimental evaluations demonstrated that a single GRU layer is optimal for capturing these sequential patterns; adding additional recurrent or feedforward layers actively degraded performance, likely due to the short lifespan and sparse nature of typical user sessions. Ultimately, the network’s final layer outputs a predictive likelihood score for every item in the catalog, estimating the exact probability of each item being the user’s next interaction.

Based on the success of RNNs, Multi-hop Time-aware Attentive Memory (MTAM) [32] was proposed in 2020. A novel framework designed to improve sequential recommendation by better modeling user behavior. The authors were one of the few to introduce the temporal information in their method and particularly in an RNN-based approach. These models overcome the limitations of traditional deep learning architectures by incorporating a temporal gating mechanism that accounts for the time intervals between user interactions into both recurrent units and attention mechanisms. By utilizing a specialized T-GRU to capture immediate user intent and a multi-hop memory reader to access long-term preferences, the system creates a more comprehensive representation of a user’s evolving interests. This approach treats short-term intent as a query to filter historical records, effectively balancing long-term memory with current intent. Extensive testing on six real-world datasets demonstrates that MTAM significantly outperforms state-of-the-art methods in Top-K recommendation tasks. Highlighting how incorporating temporal dynamics generally improves the recommendation of the model, being crucial for accurately predicting future actions in different domains.

3.5 Graph Neural Network Approaches

More recently, Graph Neural Networks (GNNs) have emerged as a dominant framework for session-based recommendation. This approach generally involves modeling user sequences as

session graphs and then applying GNNs to learn item representations based directly on the topological structure of the data. The primary advantage of GNNs over traditional deep learning architectures, such as standard Feed Forward Neural Networks, is their ability to capture complex, high-order structural dependencies through neighborhood aggregation. In modern literature, these GNN architectures are typically followed by an attention layer, which aggregates the refined item embeddings to construct the final, unified session representation.

A pioneering work in this domain is SR-GNN. In 2019, Wu et al. proposed the use of Graph Neural Networks (GNNs) for session-based recommendation to address the limitations of conventional sequential models, such as RNNs, which struggle to capture complex item transitions beyond simple linear ordering. The authors hypothesized that GNNs can take advantage of structural dependencies by modeling sessions as directed graphs, where items act as nodes and edges represent sequential transitions.

In this architecture, node embeddings are learned by propagating information across the session graph using a variant called Gated GNN, which relies on a GRU-based function to update the node states. Once the item embeddings are obtained from the GNN, the model generates a comprehensive session representation by dynamically balancing a user's long-term and short-term preferences.

The user's short-term interest, or local embedding, is represented directly by the latent vector of the last clicked item in the session. To capture the long-term preference, or global embedding, a soft-attention mechanism is applied over all items involved in the session. This mechanism computes an attention score for each item by measuring its relevance against the final clicked item, which acts as a query. The global session representation is then calculated as a weighted sum of all the item embeddings based on these attention scores.

Finally, a hybrid session representation is constructed by concatenating the local and global representations. A linear transformation is then applied to compress these combined embeddings back into the original latent space dimension, which is ultimately used for the next-item prediction.

In 2020, the authors of [?] proposed the Target Attentive Graph Neural Network (TAGNN) for session-based recommendation. They hypothesized that because candidate items are abundant and user interests are highly diverse, representing an entire session with a single embedding vector limits the model's capability. To address this, their proposed model adaptively activates specific user interests by evaluating the relevance of a user's historical behaviors against a given target item.

To achieve this, the authors introduced a local target attentive layer. This mechanism computes attention scores between all the items within the current session and every potential candidate item to be predicted. Specifically, a shared non-linear transformation is applied to each item-target pair, and the resulting scores are then normalized using a softmax function. Finally, the session's representation—tailored specifically to a given target item—is computed as

a weighted sum of the session’s items based on these normalized attention scores.

In 2022, the authors of Disen-GNN [?] challenged the conventional assumption that an item possesses a single, static representation. They argued that a user’s behavior during a session is highly dynamic, and the motivation for clicking an item is often driven by diverse latent factors, such as brand, price, or style. Consequently, two items might be highly similar regarding one factor but completely different regarding another.

To address this, the authors utilized disentangled representation learning, proposing that items in the latent space should be represented as a combination of distinct features. In this approach, the embedding of each item is divided into multiple independent chunks, with each chunk representing a specific, isolated latent factor. The initial representation of an item is then formed by concatenating these distinct factor chunks.

Because items can share similarities in specific factors while differing in others, the authors defined a factor-based similarity mechanism. This approach calculates the similarity between adjacent items in a session independently for each latent factor. These factor-specific similarity weights are then used to construct an adjacency matrix for the session graph. This graph is subsequently fed into a Graph Neural Network—similar to earlier baseline approaches—but with key architectural modifications. For instance, a distance correlation function is introduced as a regularizer in the first GNN layer to explicitly measure and encourage statistical independence among the learned factors of the items.

Additionally, to prevent items from losing their distinct characteristics as information propagates through the network—a common issue known as over-smoothing—the authors integrated a residual attention mechanism. Finally, to generate the ultimate session representation, the model dynamically combines both local and global user preferences to filter out noise and accurately capture the user’s intent. The local intent is inferred directly from the final clicked item in the sequence, while the global intent aggregates the broader context of the entire session, which are then combined to predict the user’s next interaction.

CHAPTER 4

SOLUTION APPROACH

4.1 Notation

Scalars are denoted using lower-case letters, vectors by bold lower-case letters, matrices using bold upper-case letters, and sets by upper-case letters. Table 4.1 summarizes the main notation used throughout this work.

Table 4.1: Notation table

Expression	Meaning
V	Set of all items
S	Set of all sessions
s	Anonymous session sequence
v_i	Item interaction at position i in a session
$G_s = (V_s, E_s)$	Session graph corresponding to session s
V_s	Set of nodes (unique items) in the session graph
E_s	Set of edges in the session graph
A	Undirected adjacency matrix
A_{in}, A_{out}	Incoming and outgoing adjacency matrices
$\tilde{A}_{in}, \tilde{A}_{out}$	Row-normalized incoming and outgoing adjacency matrices
h_t^i	Hidden embedding of item i at layer/time step t
a_t^i	Aggregated neighborhood representation of item i
d	Embedding dimensionality
W, W_*	Learnable transformation matrices
b, b_*	Learnable bias vectors
z_t	Update gate
r_t	Reset gate
$\tilde{h}_s$	Candidate hidden representation
$\odot$	Hadamard (element-wise) product
σ	Sigmoid activation function
ϕ	Non-linear activation function
H	Matrix of item embeddings in the session
Q, K, V	Query, Key and Value matrices in the attention mechanism
T	Time-interval matrix
δ	Temporal correlation representation
τ	Structural correlation representation
G	Time-aware gate matrix
$\hat{h}_i$	Attention-updated embedding of item i
s_g	Global session representation
s_l	Local session representation
$s_{f_{se}}$	Final sequential session representation
$s_{f_{st}}$	Final structural session representation
$logits_{se}, logits_{st}$	Sequential and structural item logits
$scores$	Final predicted probabilities over items
L	Cross-entropy loss function
$\hat{y}$	Predicted probability distribution
y	Ground truth one-hot vector
λ_{hit}	Number of hits within Top- K recommendations
$RR@K$	Reciprocal Rank at K
$MRR@K$	Mean Reciprocal Rank at K

4.2 Proposed architecture overview

An end-to-end architecture is proposed based on graph neural networks and an attention mechanism.

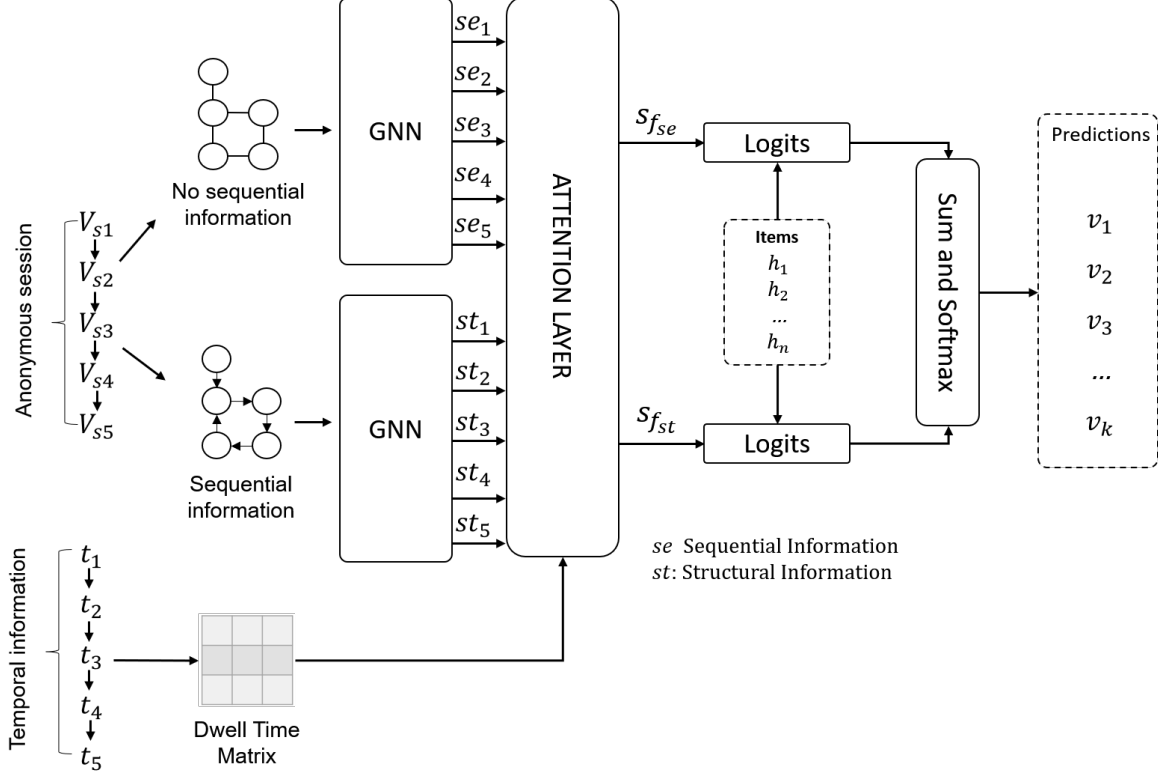


Figure 4.1: High-level diagram of the proposed architecture

For sequential modeling, a gated graph neural network is proposed, similar to [26] and [32]. As mentioned in equations [cite theory], message and update functions are proposed as follows:

4.3 Non-sequential Modeling

Let $s = [v_1, v_2, v_3, \dots, v_n]$ be an arbitrary, anonymous session sequence, containing a series of user interactions v_i ordered by timestamps, where each v_i belongs to the unique set of items V . To capture the topological structure of these interactions, the session sequence is mapped to a session graph $G_s = (V_s, E_s)$. Here, the vertex set $V_s = \{u_1, u_2, \dots, u_m\}$ represents the unique items present in s ($m \leq n$), and E_s denotes the connections between consecutive items. From this graph representation, we construct an undirected matrix that captures item co-occurrence independently of the order of the events in the session

The undirected matrix, denoted as $\mathbf{A} \in \mathbb{R}^{m \times m}$, is constructed as a binary matrix in a symmetric fashion. If a transition occurs in either direction between an item u and an item v within the

sequence s , a bidirectional link is established in the graph structure. Figure 4.2 shows an example of a session, the undirected session graph, and the respective adjacency matrix $\mathbf{A}$.

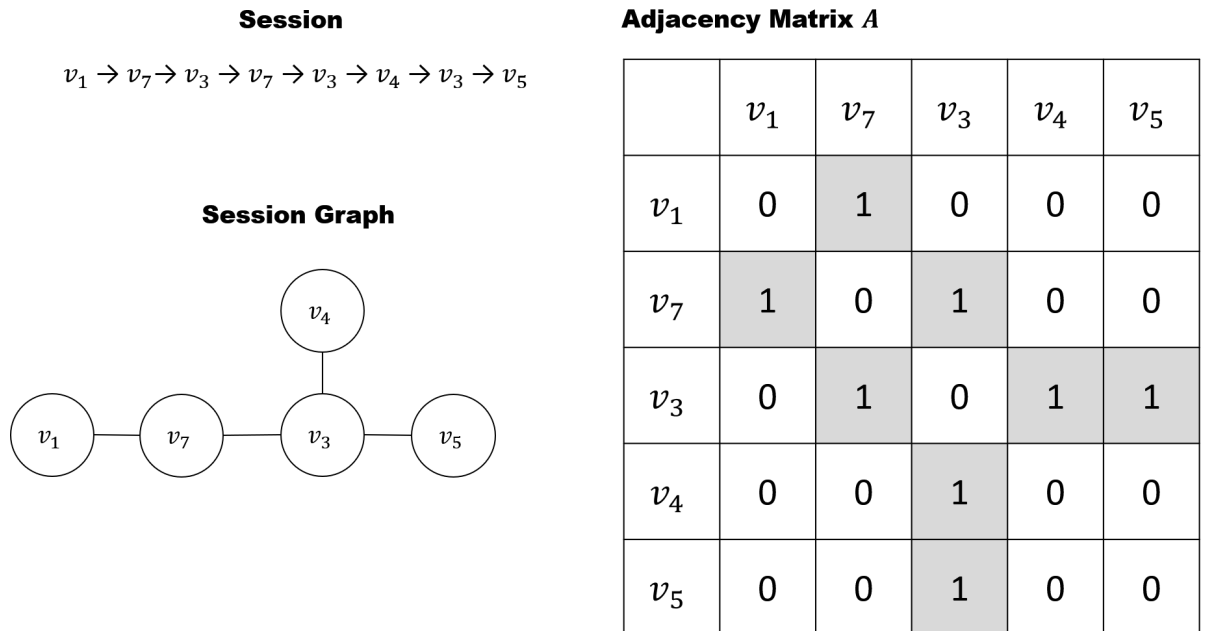


Figure 4.2: A toy example of a session containing 8 interactions with 5 unique items: from item v_{s1} to item v_{s5} . The respective undirected session graph is constructed, indicating connections between items in the session. And the respective symmetric adjacency matrix $\mathbf{A}$ is presented.

To perform message passing over the non-sequential graph representation, a proposed Graph Neural Network (GNN) layer aggregates neighborhood structures using the unnormalized, symmetric undirected matrix $\mathbf{A}$. This layer maps structural item dependencies into continuous latent vectors through a message aggregation phase and a gated update mechanism.

Message Aggregation Phase The message function is designed to collect structural features from the immediate neighborhood of an item. Let $\mathbf{h}_{t-1}^i \in \mathbb{R}^d$ denote the embedding of item v_i from the previous layer ($t-1$), where d represents the embedding dimensionality. To extract the spatial context for a specific item v_i , the model computes the aggregated neighborhood vector $\mathbf{a}_t^i \in \mathbb{R}^d$ by slicing the i -th row of the session's directed adjacency matrix $\mathbf{A}$:

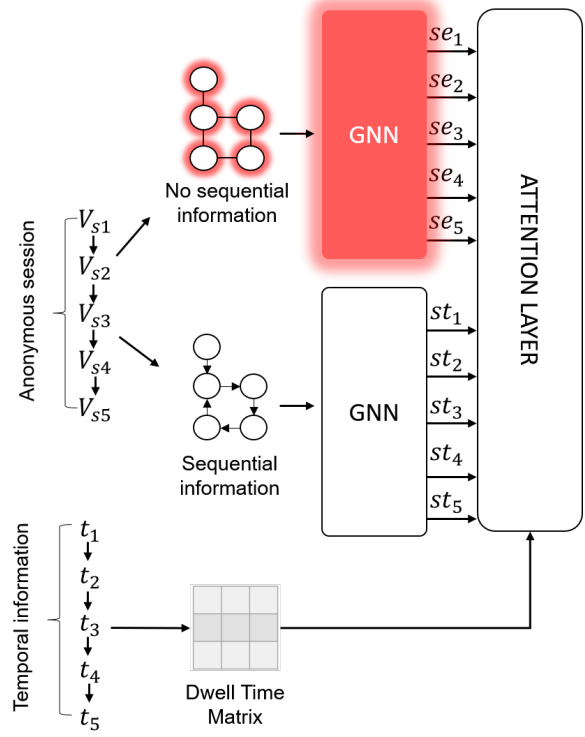


Figure 4.3: Thumbnail view of the non-sequential layer

$$\mathbf{a}_t^i = g(\mathbf{W}[\mathbf{A}_i; \mathbf{H}]) \quad (4.1)$$

Where g is the *LeakyReLU* function, $\mathbf{W} \in \mathbb{R}^{d \times d}$ is a transformation matrix, $\mathbf{A}_i \in \mathbb{R}^{1 \times m}$ represents the i -th row of the concatenated adjacency matrix $\mathbf{A}$, corresponding to the incoming and outgoing structural edges of item v_i (structural information) and $\mathbf{H} \in \mathbb{R}^{m \times d}$ represents the matrix containing all the item embeddings in the session. This row-vector multiplication aggregates the latent features of adjacent items of item v_i .

Update Phase Once neighborhood information is aggregated, the update function controls how much of the new graph structure is integrated into the node's existing representation. First, the raw neighborhood message $\mathbf{a}_t^i$ is projected into the hidden space via a learnable weight matrix $\mathbf{W}_s$ and modulated by a non-linear activation function:

$$\mathbf{g}_1 = \sigma(\mathbf{W}[\mathbf{a}_t^i; \mathbf{h}_{t-1}^i]) \quad (4.2)$$

$$\mathbf{h}_t = \mathbf{g}_1 \odot \mathbf{h}_{t-1}^i + (1 - \mathbf{g}_1) \odot \mathbf{h}_{t-1}^i \quad (4.3)$$

$$h_t = g_1 \odot h_{t-1}^i + (1 - g_1) \odot h_{t-1}^i \quad (4.4)$$

To determine the balance between a node’s historical features and its new structural context, a gating mechanism $\mathbf{g}_1$ is computed. This gate uses a sigmoid activation function σ over the concatenation of the transformed message and the previous hidden states $\mathbf{h}_{t-1}^i$, scaled by a trainable shared weight matrix $\mathbf{W}$:

Finally, the updated node embeddings $\mathbf{h}_s$ are obtained by executing an element-wise interpolation between the previous state and the aggregated context, gated by $\mathbf{g}_1$: where $\odot$ denotes the Hadamard (element-wise) product. This non-sequential update configuration forces the GNN to refine item embeddings based purely on spatial proximity and concurrent structural contexts inside the session graph.

4.4 Sequential Modeling

To preserve the direction of the user’s navigation behavior, a directed matrix $\mathbf{A}_{\text{out}} \in \mathbb{R}^{m \times m}$ is constructed. In this matrix, m is the number of unique items in the session, and the rows and columns represent the source and target items, respectively. Therefore, a connection (u, v) in the matrix $\mathbf{A}_{\text{out}}$ indicates that node v was visited after node u as a 1-hop neighbor, so there is an outgoing link from u to v . $\mathbf{A}_{\text{out}}[u, v] = 1$ if $u \rightarrow v$.

Based on this core idea, the information flow is modeled in two distinct directions: outgoing connections and incoming connections. For incoming links, The matrix tracking incoming connections is formally defined as the transpose of the matrix $\mathbf{A}_{\text{out}}$, $\mathbf{A}_{\text{in}} = \mathbf{A}_{\text{out}}^T$ and each edge $(u, v) \in \mathbf{A}_{\text{in}}$ represents an incoming link, meaning that node v was reached from node u as a 1-hop neighbor.

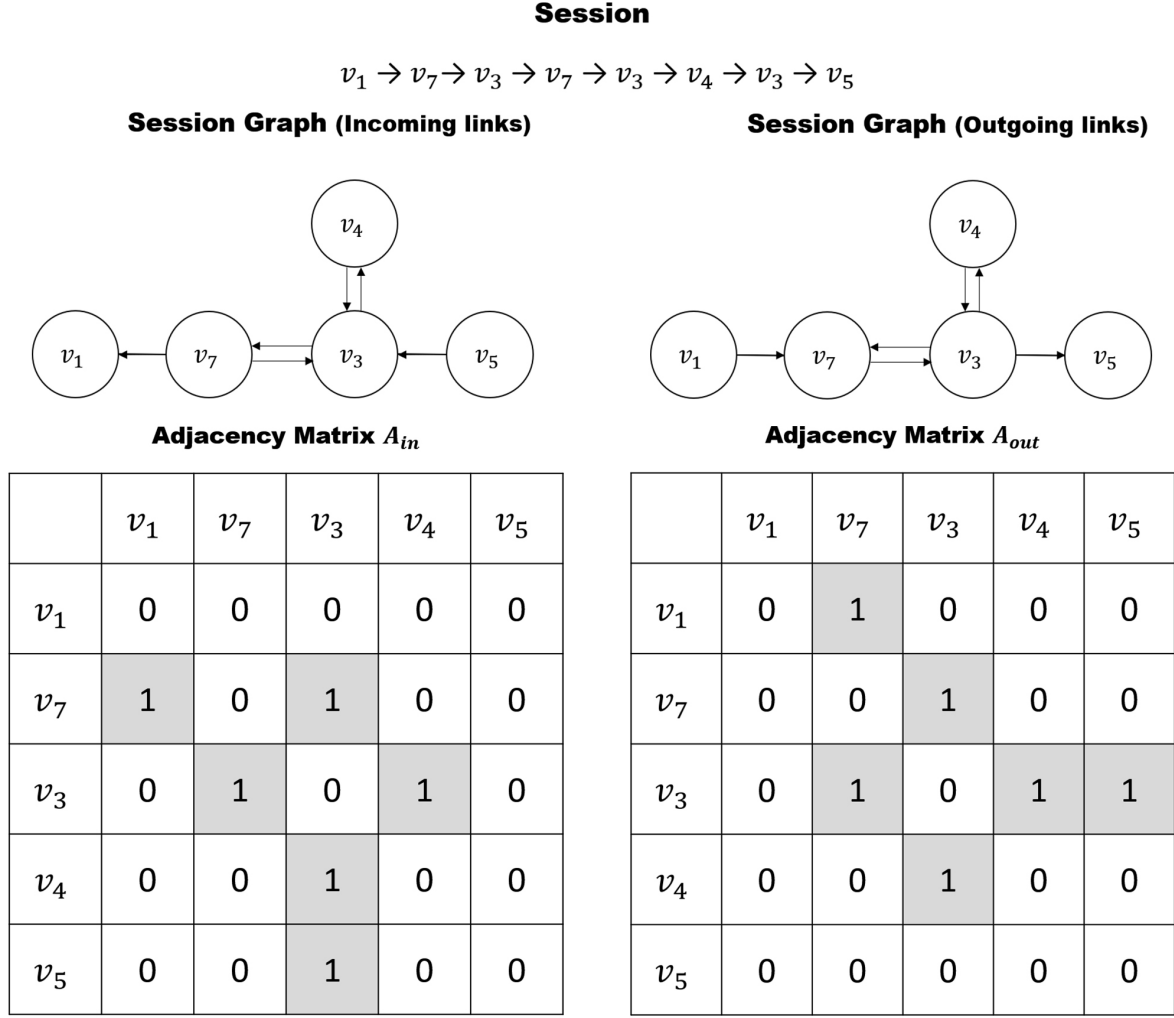


Figure 4.4: Example of sequential modeling, the example illustrates an anonymous session and their respective session graphs. The graphs model incoming and outgoing connections with their respective adjacency matrices A_{in} and A_{out}

To mitigate the impact that items with high transition frequencies (hub nodes) have during the GNN message-passing phase, both matrices are normalized independently based on their row degrees. The out-degree $d_{out}(u)$ and the in-degree $d_{in}(u)$ for each node u are computed as the sum of the elements along their respective rows:

$$d_{out}(u) = \sum_{v=1}^m A_{out}[u, v], \quad d_{in}(u) = \sum_{v=1}^m A_{in}[u, v] \quad (4.5)$$

The row-normalized matrices, $\tilde{A}_{out}$ and $\tilde{A}_{in}$, are obtained by dividing each transition by the corresponding node's degree. Potential zero-division errors for nodes without incoming or outgoing

links are bypassed by forcing a default value of 1 for the degree:

$$\tilde{A}_{out}[u,v] = \frac{A_{out}[u,v]}{\max(d_{out}(u), 1)}, \quad \tilde{A}_{in}[u,v] = \frac{A_{in}[u,v]}{\max(d_{in}(u), 1)} \quad (4.6)$$

The figure x shows the normalized matrices presented in figure 4.4

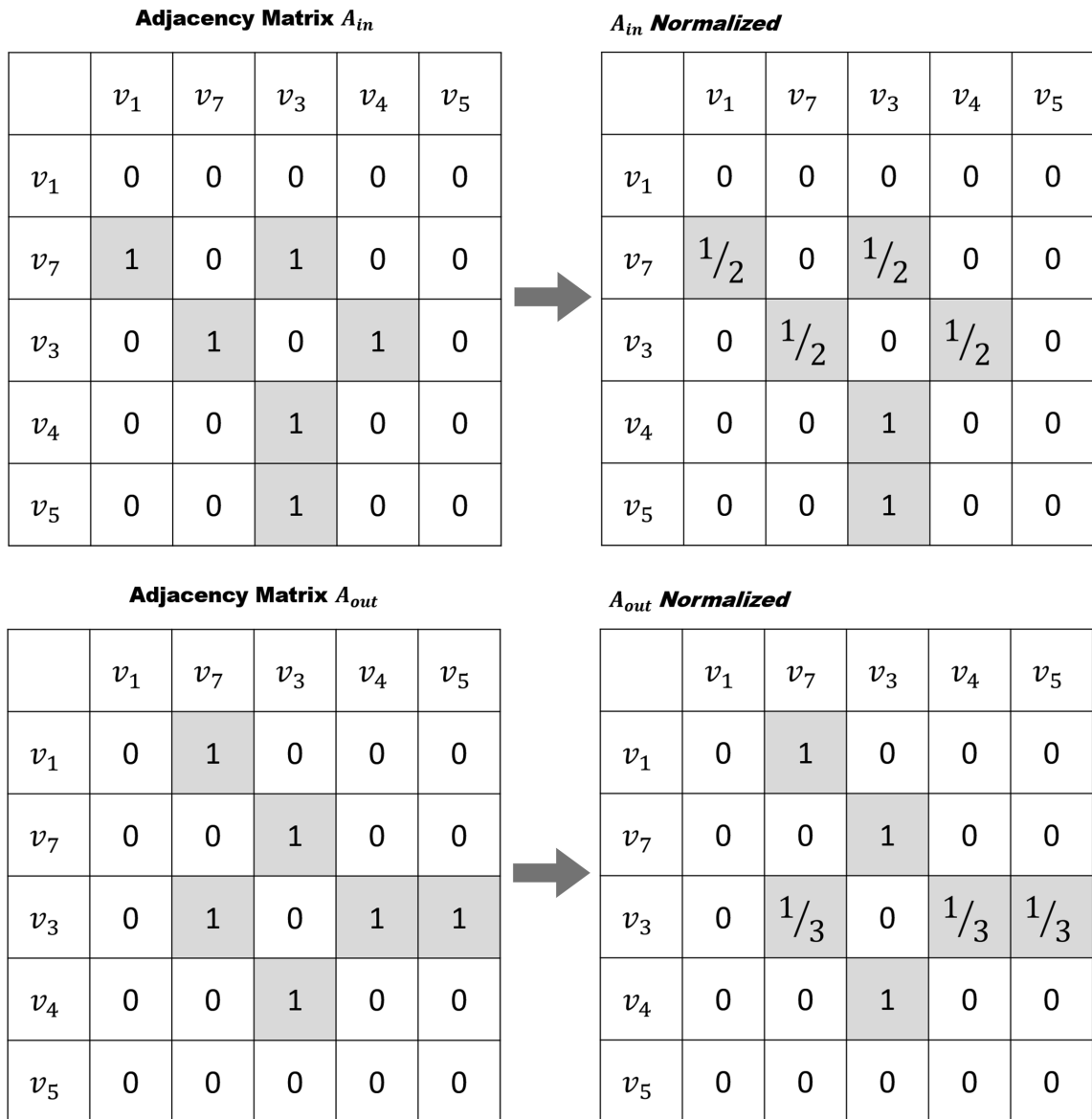


Figure 4.5: Matrices A_{in} (left) and A_{out} (right) and their respective normalized matrices $\tilde{A}_{in}$ and $\tilde{A}_{out}$.

Message Aggregation Phase

For sequential modeling, the core idea is to keep track of the incoming and outgoing connections in the sequence during the message-passing step, to achieve this behavior, both adjacency matrices $\tilde{\mathbf{A}}_{in}$ and $\tilde{\mathbf{A}}_{out}$ as described in figure 4.5 are considered, then, this process is defined formally as follows:

$$\mathbf{a}_t^i = g[\mathbf{W}_{in}(\tilde{\mathbf{A}}_{in;i}\mathbf{H}), \mathbf{W}_{out}(\tilde{\mathbf{A}}_{out;i}\mathbf{H})] \quad (4.7)$$

The update phase is done using a GRU mechanism as follows:

$$\mathbf{z}_t = \sigma([\mathbf{a}_t^i, \mathbf{h}_{t-1}^i]\mathbf{W}_z + \mathbf{b}_z) \quad (4.8)$$

$$\mathbf{r}_t = \sigma([\mathbf{a}_t^i, \mathbf{h}_{t-1}^i]\mathbf{W}_r + \mathbf{b}_r) \quad (4.9)$$

$$\tilde{\mathbf{h}}_s = \tanh(\mathbf{a}_t^i, (\mathbf{r}_s \odot \mathbf{h}_{t-1}^i)\mathbf{W}_h + \mathbf{b}_h) \quad (4.10)$$

$$\mathbf{h}_s = \mathbf{z}_s \odot \mathbf{h}_{t-1}^i + (1 - \mathbf{z}_s) \odot \tilde{\mathbf{h}}_s \quad (4.11)$$

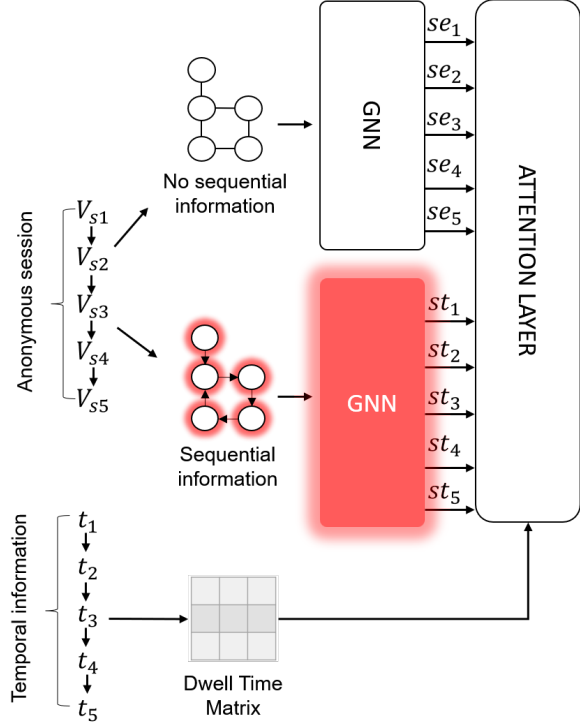


Figure 4.6: Thumbnail view of the sequential layer

We define sequential modeling as the structural information without taking into consideration the order of the items in the session. Similarly to sequential modeling, we use a graph neural network framework to obtain node embeddings for each node in the session. For the update, we propose a slightly different function, following the gating idea, the message and update functions are given as follows:

4.5 Attention Layer

Once the embeddings for session items are computed via Graph Neural Networks, an attention mechanism is used to capture the importance of all items in the sessions. And then, the final session representation is obtained by computing a compact representation of the whole session by simply adding up all item embeddings. Similar to [33], scaled-dot-product attention is used:

$$\text{Scores} = \frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}} \quad (4.12)$$

Where $\mathbf{Q} = \mathbf{W}_Q\mathbf{H}$ and $\mathbf{K} = \mathbf{W}_K\mathbf{H}$, being $\mathbf{H}$ the item embeddings matrix of the current session. $\mathbf{W}_Q$ and $\mathbf{W}_K$ are transformation matrices to project current embeddings in the latent space. The scores are divided by $\sqrt{d}$ to avoid large values of the inner product, especially when d (the size of the hidden dimension of the vectors in the latent space, which is a scalar) is high.

Time aware component

In order to let the attention mechanism be time-aware using the temporal information in the dataset, similar to [32], the idea is to take the time component into account when computing the correlation between 2 items. First, a time-interval matrix is computed and defined as follows:

$$\text{Scores} = \frac{\mathbf{Q}\mathbf{K}^T \odot \mathbf{G}}{\sqrt{d}} \quad (4.13)$$

A matrix $\mathbf{T}$ is defined as $\mathbf{T} = \mathbf{H}^\wedge \mathbf{H}$, where $\mathbf{H} \in \mathbb{R}^{l \times d}$ such that $T_{i,j} = |H_i - H_j|$ and $\mathbf{T} \in \mathbb{R}^{n \times n}$. A time gate matrix is computed as follows:

$$\boldsymbol{\delta} = \phi(\log(\mathbf{T}) + 1) \odot \mathbf{W}_\delta + \mathbf{b}_\delta \quad (4.14)$$

$$\boldsymbol{\tau} = \phi(\mathbf{W}_t\mathbf{H} + \mathbf{b}_\tau) \quad (4.15)$$

$$\mathbf{G} = \sigma(\boldsymbol{\delta} \odot \mathbf{W}_{G_\delta} + \boldsymbol{\tau} \odot \mathbf{W}_{G_\tau} + \mathbf{b}_g) \quad (4.16)$$

The intuition of the time gate $\mathbf{G}$ is to somehow learn the importance between temporal and structural information. $\boldsymbol{\delta}$ captures the importance of the temporal information for each pair of correlated items according to their time span in the session. Similarly, $\boldsymbol{\tau}$ captures the structural

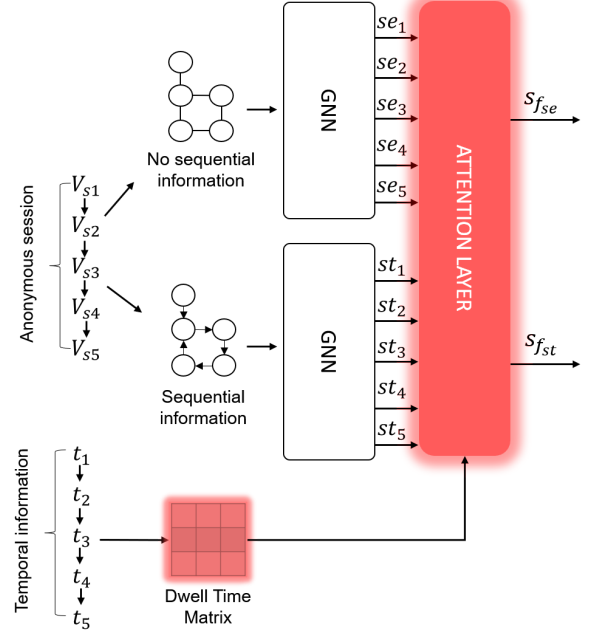


Figure 4.7: Thumbnail view of the attention layer

information of the session. Finally, the gate $\mathbf{G}$ computes the final trade-off between temporal and structural information using a linear interpolation between δ and τ . Once the time gate $\mathbf{G}$ is computed, the attention scores described in equation 4.12 are updated as follows:

$$\text{Scores} = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T \odot \mathbf{G}}{\sqrt{d}}\right) \quad (4.17)$$

Where $\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{n \times d}$ and $\mathbf{G} \in \mathbb{R}^{n \times n}$. Then, the final attention is computed using equation 4.17 as follows:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T \odot \mathbf{G}}{\sqrt{d}}\right) \mathbf{V} \quad (4.18)$$

Where $\mathbf{V} \in \mathbb{R}^{n \times d}$ is simply the matrix $\mathbf{H}$ of the items in the session and $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \in \mathbb{R}^{n \times d}$ contains the weights for all items in the session.

Using the outputs from Equation 4.18, the final session representation is constructed to capture the user's intent. To capture this representation, we extract either the global or the local interest of the session. For this, let the sequence of updated item embeddings obtained from the attention layer be denoted as $[\hat{\mathbf{h}}_1, \hat{\mathbf{h}}_2, \dots, \hat{\mathbf{h}}_n]$.

The first approach captures the global interest ($\mathbf{s}_g$), which models the user's overall preference throughout the entire session. This is achieved by applying a mean pooling operation over all the updated item representations in the sequence:

$$\mathbf{s}_g = \frac{1}{n} \sum_{i=1}^n \hat{\mathbf{h}}_i \quad (4.19)$$

where n represents the total number of items in the current session.

The second approach considers only the local interest ($\mathbf{s}_l$). This strictly captures the user's most recent, short-term intent, which is represented directly by the latent vector of the very last interacted item in the sequence:

$$\mathbf{s}_l = \hat{\mathbf{h}}_n \quad (4.20)$$

Because our architecture independently processes both sequential and non-sequential (structural) signals, as shown in figure 4.5, this process is applied to both pathways. For simplicity and clarity in the following formulations, we denote the overall final session representations regardless of whether they were computed using the local or global approaches as $\mathbf{s}_{f_{se}}$ for the sequential signal and $\mathbf{s}_{f_{st}}$ for the structural signal.

4.6 Prediction Layer

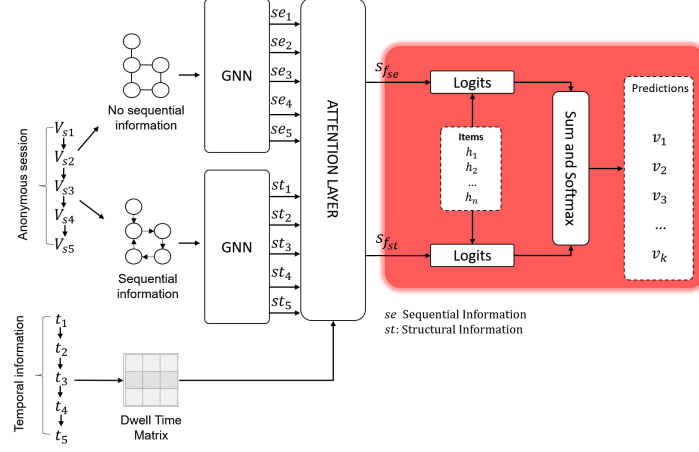


Figure 4.8: Thumbnail view of the prediction layer

Using the final representations of the sessions $s_{f_{se}}$ and $s_{f_{st}}$, the final prediction is obtained. From each session representation, item logits are computed as follows:

$$\text{logits}_{se}(V, s_{f_{se}}) = V s_{f_{se}} \quad (4.21)$$

$$\text{logits}_{st}(V, s_{f_{st}}) = V s_{f_{st}} \quad (4.22)$$

Where $s_{f_{se}}$ and $s_{f_{st}} \in \mathbb{R}^d$ and $V \in \mathbb{R}^{n \times d}$. To obtain the final item scores, logits are added up and softmax function is applied:

$$\text{scores} = \text{Softmax}(\text{logits}_{se} + \text{logits}_{st}) \quad (4.23)$$

Finally, for the final recommendation, scores are sorted and the first top K items are selected.

4.6.1 Model Evaluation

For evaluation, 2 metrics were considered: *Precision@K* (also known as *Hit Rate*) and Mean Reciprocal Rank *MRR@K*.

Precision@K is defined as follows:

$$\text{Precision@K} = \frac{\lambda_{hit}}{N} \quad (4.24)$$

This metric captures model precision by counting the number of times the target element was within the *TOP K* recommendations. λ_{hit} represents the number of times the prediction was in the top-20 recommendations among the whole batch of sessions. Being N the number of sessions

in the batch. $MRR@20$ is defined as follows:

$$RR@K = \frac{1}{rank(i)} \quad (4.25)$$

$$MRR@K = \frac{1}{N} \sum_{i \in Rec}^N RR@K \quad (4.26)$$

$MRR@20$ evaluates the quality of the recommendation and takes into account the position of the recommended items. i represents the target item. And $rank(i)$ tells us the position of the item in the $TOP K$ recommendations. $RR@K$ is the reciprocal of the rank. And $MRR@K$ is the mean of the reciprocal rank across N different sessions. If the target item is not in the recommendation list, $RR@K = 0$.

4.6.2 Loss function

For any given session, the categorical cross-entropy loss function between the predictions and the ground truth is used:

$$L(\hat{\mathbf{y}}) = - \sum_{i=1}^{|V|} y_i \log(\hat{y}_i) \quad (4.27)$$

Where $\mathbf{y}$ denotes the ground truth one-hot vector, $\hat{\mathbf{y}}$ is the vector of predicted probabilities for all V items and $\hat{y}_i$ is the probability assigned to item i . Since the ground truth for each session is a one-hot encoded vector all elements y_i are zero except for the one corresponding to the actual next item in the session ($y_j = 1$). Consequently, Equation 4.27 simplifies to the negative log-likelihood of the true item:

$$L(\hat{\mathbf{y}}) = -\log(\hat{y}_j) \quad (4.28)$$

where $\hat{y}_j$ represents the predicted probability for the ground truth item j .

EXPERIMENTAL RESULTS

5.1 Data

The proposed model was evaluated on three representative real-world datasets for session-based recommendation: Diginetica, Yoochoose 1/64 (a standard fraction of the massive Yoochoose dataset) and Nowplaying. The first two contain information from e-commerce, while the last one comes from the domain of music. Diginetica is a retail technology company. The Diginetica dataset is from the CIKM 2016 Cup and is publicly available at this link. It has 573,935 sessions, 134,319,529 products, 18,025 purchases, and contains TimeStamp information between interactions. The Yoochoose dataset provides a record of six months of user activity from a major European e-commerce platform, covering a vast catalog of products, including garden tools, toys, clothes, electronics, and more. The dataset is from the RecSys Challenge 2015, it contains a collection of sequences of click events, click sessions, and purchase events. It contains 9,249,729 sessions, 52,739 products, 1,150,753 purchases, and TimeStamp information. The Nowplaying dataset comes from the domain of the music. It contains data from users who publish the music they were currently listening to on Twitter (Now X). It includes tracks which have been tweeted using hashtags like nowplaying, listento or listeningto. For a fair comparison with other baselines, for all datasets, sessions of length 1 and items that appear fewer than 5 times are excluded. To allow our model to potentially generalize better, data augmentation is performed, generating sets of sub-sessions. Given a session $s = [v_{s1}, v_{s2}, \dots, v_{sn}] \in S$, multiple sequence-target pairs are constructed through the following process: $\{([v_{s1}, \dots, v_{si}], v_{s,i+1}) \mid 1 \leq i < n\}$. The final statistics of the 3 datasets after pre-processing are summarized in table 5.1:

Table 5.1: Statistics of Yoochoose 1/64, Diginetica, and Nowplaying Datasets after preprocessing

Statistics	Yoochoose 1/64	Diginetica	Nowplaying
# Clicks	557,248	982,961	1,367,963
# Training sessions	369,859	719,470	825,304
# Test sessions	55,898	60,858	89,824
# Items	16,766	43,097	60,417
Avg. length	6.16	5.12	7.42
Max length	146	70	30

5.2 Time analysis

To analyze the time information contained in each dataset. The idea is to see the distribution of times in all datasets. Hence, a natural idea is to count time frequencies. For each dataset, in order to compute the time distribution, frequencies are counted by discretizing the time spans between items. Let $T_s = [t_1, t_2, \dots, t_n]$ represent the timestamps for each item in a sequence s . To compute the dwell times for each session, the difference between timestamps t_{i+1} and t_i is computed as $\Delta_i = t_{i+1} - t_i$. Since timestamps present in the datasets and dwell times are usually large integers or even real numbers, $\log$ normalization was used. Formally, $\tilde{\Delta} = \text{round}(\log(\Delta + 1))$ and the round function is applied to get discrete values which we have called *bins*. The larger the bin, the larger the dwell time between items in a session. Figure 5.1 shows the results after computing the dwell time's frequencies for each dataset using log normalization to get discrete bins and see the time distribution across all datasets.

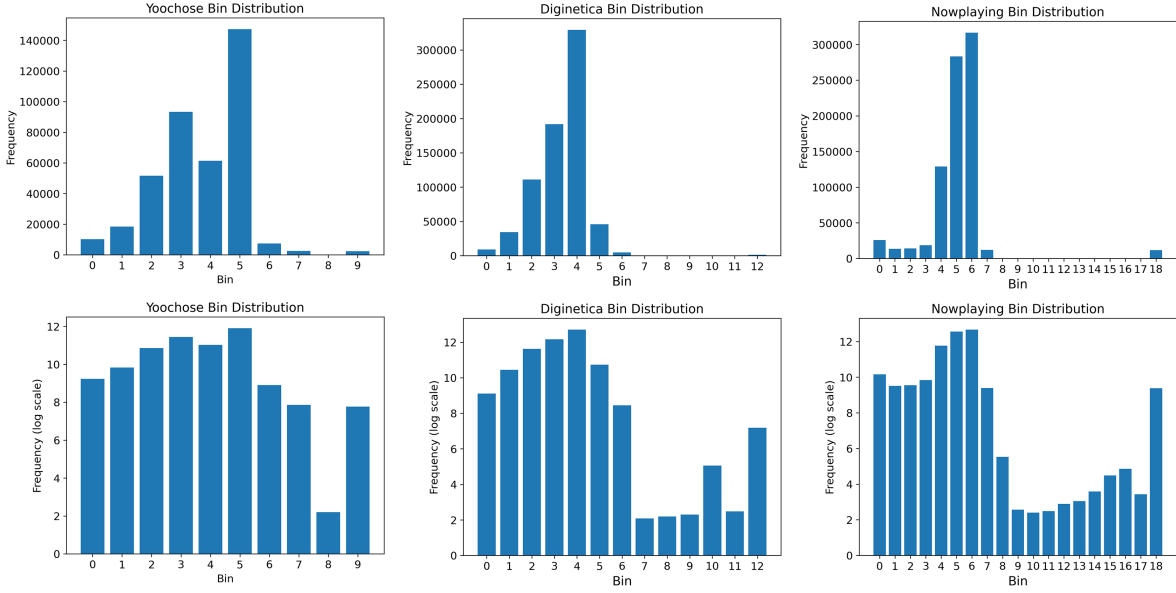


Figure 5.1: Analysis of the temporal information on the three datasets. For every chart, the horizontal axis indicates a discrete bin (obtained from the logarithm of the dwell times computed between the items in all sessions from the dataset), and for the charts in the top, the vertical axis, represents the frequency for each bin computed across the entire dataset. For charts in the bottom, the vertical axis is the frequency of each bin using a logarithmic scale too. The pictures are placed from left to right, where Yoochoose is the dataset containing the smallest dwell times and Nowplaying containing the largest dwell times. Since this last dataset comes from the music domain, it contains larger dwell times between items since songs are usually larger events than interactions the e-commerce domain.

5.3 Experimental setup

For all the experiments, a hidden size $d = 100$ (representing the dimension of item embeddings in the latent space) and a batch size of 100 were used. An learning rate (lr) starting at 0.001 with learning rate decay of 0.1 every 3 steps was used. l_2 penalty $l_2 = 1 \times 10^{-5}$ was used as a regularization term to prevent overfitting and potentially improve generalization. The number of epochs was set to 30 and a patience parameter was used as an early stopping criterion.

For evaluation, $Precision@K$ (also known as *Hit Rate*) and Mean Reciprocal Rank $MRR@K$ were considered. All the parameters of the model were normalized with a mean of 0 and a standard deviation of 0.1. All the experiments were executed 3 times with 3 different seeds (from 40 to 42) and the mean of the results is reported.

5.4 Results obtained

Table 5.2: Combined results for Diginetica, Yoochoose, and Nowplaying datasets

Description	Diginetica		Yoochoose		Nowplaying
	P@20	MRR@20	P@20	MRR@20	MRR@20
Hybrid	53.0338	18.5841	71.6436	31.1399	8.0564
Hybrid + Time Gate	53.1209	18.6351	71.5750	31.0830	8.1571
Only Sequential	52.5497	18.4186	71.2160	30.3928	7.6119
Only Sequential + Time Gate	52.4763	18.3627	71.1630	30.2838	7.5357
Only Non-Sequential	51.8798	17.4643	71.1254	31.2610	8.0580
Only Non-Sequential + Time Gate	51.8913	17.5536	71.1266	31.1471	8.2062

RQ1 (Ablation study): What is the effect of sequential and non-sequential information on the recommendation performance?

Sequential and non-sequential experiments (isolated) As shown in the results, to evaluate the effect of sequential and non-sequential information on the recommendation performance, we isolate the effect of these two signals. The experiments "Only Sequential" and "Only Non-Sequential" help us to answer this question. According to the results, for Diginetica, best results are obtained using Only sequential information on both metrics, but also, good results are obtained using only Non-sequential information. On the other hand, for Yoochoose, results obtained for $P@20$ are very similar in both experiments, while for $MRR@20$, best results are obtained using the Non-sequential information. Finally, for Nowplaying, similar results are obtained on both experiments. Since good results are obtained on all experiments described above, we conclude that both signal captures useful information from the session, and that the preference of one signal over the other will depend on the information of the dataset. The natural next step is to analyze the combination of these two signals (as explained in the previous chapter) and the Hybrid experiments in the table represent this study. Where we can see that in almost all cases, this hybrid approach is superior, which re-emphasizes the importance of both signals for session-based recommendation.

RQ2 (Ablation study): What is the effect of temporal dynamics in the final recommendation? Does the effect depend of the nature of the temporal information in the data? For Only Sequential experiments, using the time component (Time gate), results show similar performance that not using it, but in almost all datasets, the best results are obtained by avoiding the time gate. In contrast, for experiments of only non-sequential information, even the performance improvement is just slightly better, the best results are obtained by using the time gate.

When using the time component for the hybrid experiments, the best results overall are obtained in the Diginetica and Nowplaying datasets. And we argue that the temporal information appears to be useful when transitions of long duration are present in the data. Which is actually the case for Diginetica and Nowplaying. Therefore, we conclude that the effect of temporal dynamics is

dataset-dependent.

Table 5.3: Dataset Statistics for Short and Long Sequences

Dataset		Short	Long
Nowplaying	# training sessions	628,325	68,213
	% training sessions	90.2	9.79
	# test sessions	196,979	21,611
	% test sessions	90.11	9.88
Diginetica	# training sessions	544,983	174,487
	% training sessions	75.74	24.25
	# test sessions	46,945	14,363
	% test sessions	76.57	23.42
Yoochoose	# training sessions	302,832	67,027
	% training sessions	81.87	18.12
	# test sessions	41,920	13,978
	% test sessions	74.99	25

Table 5.4: Short and Long Sessions

Experiment ID	Description	P@20	MRR@20
short_now_a	Hybrid	21.3647	8.0564
long_now_a	Hybrid	21.3198	8.0681
short_dig_a	Hybrid	52.3956	18.8165
long_dig_a	Hybrid	42.8276	14.3372
short_yoo_a	Hybrid	72.5358	32.5297
long_yoo_a	Hybrid	63.7383	24.1054

RQ3: What is the performance of the model for short and long sessions?

To evaluate the performance of the architecture for short and long sessions, we utilized the best configuration obtained in table 5.2, which was the hybrid approach. For the *Nowplaying* dataset, the model yields similar performance metrics across both short and long sessions. In contrast, for the *Diginetica* and *Yoochoose* datasets, significantly better results are obtained for shorter sessions. We attribute this disparity to the distribution of the number of sessions within the sliced datasets. It is a well-established paradigm in deep learning that larger quantities of training data usually lead to better model generalization. In the case of *Diginetica* and *Yoochoose*, the short-session subsets contain a significantly higher number of samples compared to their long-session counterparts. For *Nowplaying*, surprisingly, the model achieves comparable and balanced results across both session lengths.

Table 5.5: Dataset Evaluation Performance Metrics

Dataset	P@10	P@20	P@30	MRR@10	MRR@30
Nowplaying	16.1286	21.4382	25.0249	7.7905	8.3009
Diginetica	39.9739	53.1209	60.8433	17.7222	18.9442
Yoochoose	60.8984	71.5750	76.5036	30.2145	31.1721

RQ4: What is the performance of the model for different top- K recommendations? As illustrated in Table 5.5, the highest numeric values across all datasets are consistently obtained at $P@30$ and $MRR@30$. These experiments confirm the expected theoretical behavior of recommendation metrics when varying the cutoff parameter K . Intuitively, a larger value of K expands the model’s slack to find the target item, in other words, increasing the probability that the true target item is included in the recommended list. For instance, if a target item is ranked at the 12th position, evaluating the model strictly at $K = 10$ registers a complete failure for that sample, whereas evaluating at $K = 20$ or $K = 30$ successfully captures it. Consequently, considering the top-30 items yields the highest overall performance results.

This behavior naturally presents two theoretical extremes. At one extreme, if $K = 1$, the search space is highly restrictive, making it exceptionally difficult for the model to match the exact target item and resulting in significantly lower performance scores. Nevertheless, a single-item recommendation system ($K = 1$) remains highly relevant in practical, some applications, such as automated push notifications, smart assistant voice-responses (e.g., "Would you like to reorder your usual coffee?"), or severe screen-space constraints like smartwatches. At the opposite extreme, recommending the entire catalog of available items would guarantee a perfect precision score, however, returning the full item set is impractical, and defeats the purpose of information filtering, lacking any viable real-world application.

It is critical to note that the metric benefiting most directly from large values of K is Precision@ K , as a broader search window directly increases the mathematical likelihood of a hit. In contrast, Mean Reciprocal Rank (MRR@ K) exhibits diminishing returns as K increases. While a larger K prevents the Reciprocal Rank from dropping to absolute zero by catching deeply dug target items, items positioned far down the list contribute negligible fractional values (e.g., $1/25$ or $1/30$) to the final score. Therefore, while MRR@30 is marginally better than MRR@10, the difference is far less pronounced than the growth observed in Precision. This analysis highlights the vital importance of evaluating session-based recommender systems using both metrics simultaneously to capture both retrieval breadth and ranking quality.

5.5 Comparison with some baselines

To compare the results obtained with baseline algorithms, we select the best results obtained from table 5.2 across all datasets. We considered the following baseline algorithms for comparison:

- **POP:** Popular is a naive method in the literature that recommends the most popular item in the entire dataset.
- **S-POP:** This method is a session-based technique that recommends the most popular item in the session.
- **item-KNN:** This approach employs distance metrics (e.g., cosine similarity) to discover and recommend items that are similar to the last item in the session.
- **BPR-MF:** This approach is a matrix factorization (MF) method. Despite the fact that this algorithm is not commonly used for session-based recommendation, the latent representation of items within a session can be computed using a pairwise ranking loss. In that way, optimizing the item embeddings for the final recommendation.
- **FPMC:** It is a method based on Markov Chains, specifically suited for sequential recommendation and next-basket-prediction.
- **GRU4REC:** Is one of the first methods implementing Recurrent Neural Networks for SB-Recommendation. It implements a GRU-based model and a pairwise ranking loss function.
- **NARM:** This model is based on the transformer architecture. It implements an encoder-decoder architecture for sequence-aware recommendation [?]
- **STAMP:** This method uses a model based on the attention mechanism. The architecture tries to capture the short and long-term preferences in a sequence to improve the final recommendation.

Table 5.6 shows the results comparing experiments 4 and 7 with the baseline methods mentioned above (Best results in bold).

Table 5.6: Performance Comparison Across Different Baselines and Datasets

Type	Method	Diginetica		Yoochoose		Nowplaying	
		P@20	MRR@20	P@20	MRR@20	P@20	MRR@20
Shallow	POP	0.89	0.20	6.71	1.65	-	-
	S-POP	21.06	13.68	30.44	18.35	-	-
	Item-KNN	35.75	11.57	51.60	21.81	15.94	4.91
	FPMC	26.53	6.95	45.62	15.01	7.36	2.82
RNN	GRU4REC	29.45	8.33	60.64	22.89	7.92	4.48
	NARM	49.70	16.17	68.32	28.63	18.59	6.93
	STAMP	45.64	14.32	68.74	29.67	17.66	6.88
GNN	SR-GNN	50.73	17.59	70.57	30.94	18.87	7.47
	GC-SAN	50.80	16.87	70.89	30.98	-	-
	FGNN	51.36	18.47	71.12	31.68	18.78	7.15
	OURS	53.05	18.60	71.64	31.26	21.36	8.06

RQ5: Does the proposed method surpass traditional baselines in session-based recommendation?

According to the results presented in Table 5.6, the proposed method significantly outperforms some of the most representative shallow baseline approaches for session-based recommendation across both evaluation metrics. A similar performance advantage is observed when comparing our model to representative frameworks based on Recurrent Neural Networks (RNNs). Furthermore, regarding earlier architectures utilizing Graph Neural Networks (GNNs), our proposed method consistently achieves superior results across all evaluations and metrics.

Importantly, we highlight that our framework shares a core foundation with existing works that build upon the SR-GNN [26] approach. However, by incorporating some inductive biases—specifically, non-sequential information and the time-dynamics component, the proposed architecture captures complex session behaviors. Despite utilizing relatively simple and efficient components to model these dynamics, the proposed design effectively translates these structural biases into enhanced predictive capability, ultimately yielding superior performance over traditional baseline methods.

5.6 Comparisson with some recent methods

Table 5.7: Performance Comparison with some Recent Methods (2020–2025)

Year	Method	Diginetica		Yoochoose		Nowplaying	
		P@20	MRR@20	P@20	MRR@20	P@20	MRR@20
2020	TAGNN	51.31	18.03	71.02	31.12	-	-
	SGINM	53.91	18.59	70.89	31.33	-	-
2021	MoSeR	52.98	19.14	71.20	31.68	-	-
2022	SGNN	50.91	17.06	70.21	30.34	-	-
	SEDGN	51.91	18.22	71.42	31.15	-	-
	CORE	52.89	18.58	64.61	28.24	21.81	7.35
	SR-HGNN	52.02	18.48	71.18	31.70	-	-
	Disen-GNN	53.79	18.99	71.46	31.36	22.22	8.22
2023	Transformer4Rec	53.32	18.00	70.32	29.63	-	-
2024	TRHMCI-GNN	54.54	18.79	71.91	31.11	-	-
2025	MSERS	56.32	21.62	71.24	31.73	-	-
	GAN-GSNP	50.92	17.51	70.58	31.11	-	-
2026	OURS	53.12	18.63	71.64	31.26	21.43	8.15

RQ6: Can the proposed architecture achieve state-of-the-art results?

According to the results reported in Table 5.7, the proposed architecture achieves highly competitive performance and surpasses several recent state-of-the-art methods. For the *Diginetica* dataset, our model yields competitive results in terms of $P@20$, being noticeably outperformed only by TRHMCI-GNN and MSERS. Regarding $MRR@20$, the performance is slightly stronger, with MSERS being the only baseline that considerably outperforms our approach. For the *Yoochoose* dataset, the results demonstrate that our architecture reaches the second-best performance on $P@20$, placing just below TRHMCI-GNN. Finally, for the *Nowplaying* dataset, despite the fact that most baseline methods lack reported results on this benchmark, it can be observed that our model achieves highly competitive outcomes across both metrics, particularly in terms of $MRR@20$.

CONCLUSIONS AND FUTURE WORK

A comprehensive review of the literature on sequential recommendation was conducted. We observe that a significant portion of existing works rely on the additive attention mechanism, we were led to adapt a dot-product attention approach to evaluate its efficacy within the session-based recommendation task. Our results show that the impact of sequential and non-sequential information is dataset-dependent; nevertheless, both signals capture highly valuable information from the topological structure of the graph and the chronological order of events. Consequently, the most robust results are frequently obtained by effectively combining them.

Furthermore, we identified that time dynamics remain relatively underexplored in this specific domain. We experimented with various integration strategies, such as using dwell times as edge weights in the session graphs and making the Graph Neural Networks time-aware through a gated mechanism. Although these initial structural attempts did not yield observable performance improvements, integrating time-awareness directly into the attention layer showed promising results, enhancing the final predictive performance in many cases as reported in this work.

Overall, the empirical evaluations demonstrate that the proposed architecture achieves highly competitive outcomes. It significantly surpasses traditional baselines, including shallow techniques, Recurrent Neural Networks (RNNs), and earlier GNN-based architectures. When compared against representative state-of-the-art models, our framework demonstrates strong competency. While it does not outperform every existing model, it successfully surpasses several established methods and maintains a narrow performance gap behind some of the latest and best approaches.

Contributions

The main contributions of this work are summarized as follows:

- We propose a variation of the Gated Recurrent Unit (GRU). While structurally inspired by the mechanism utilized in SR-GNN [26], our adaptation achieves superior predictive performance across most evaluated scenarios.
- We explicitly model non-sequential item relationships, demonstrating that incorporating structural (non-sequential) graph information captures useful insights. on the overall recommendation performance.
- We introduce the use of dot-product attention specifically tailored for GNN-based recommendation architectures. Although additive attention has traditionally been the default method in this domain, our framework showed that dot-product attention yields highly competitive results, both with and without the inclusion of temporal dynamics.
- We conduct a comprehensive analysis of temporal information—an aspect that remains largely underexplored in current literature, particularly across the three benchmark datasets evaluated in this study. By mapping the interaction frequency spectrum for each dataset, we provide deeper insights into how time signals influence recommendation performance.

Future Work

We propose the following directions for future research:

- **Explore similar domains and potential applications:** Evaluate the proposed framework across a broader range of recommendation domains (such as movies, news, and diverse e-commerce datasets) to provide broader empirical evidence on non-sequential and temporal information across different scenarios. Furthermore, we aim to adapt this architecture for entirely different sequential tasks, such as time-series forecasting or processing electrical signals in medical and biological domains, to uncover potential real-world applications.
- **Creation of Global Graph:** Construct a global graph representation derived from all sessions in the dataset. This approach would allow the model to capture richer structural dependencies by leveraging a k -hop neighborhood for each item, expanding the receptive field beyond isolated session transitions.
- **Alternative approaches for temporal modeling:** Investigate alternative mechanisms to more effectively model time dynamics. Potential approaches include:
 - Refining the integration of temporal information within the attention layer, operating under the premise that items with longer dwell times have a stronger influence on a user’s final decision for the next interaction.

-
- Utilizing methodologies like Time2Vec to transform continuous dwell times into learned vector representations, thereby capturing the complex, non-linear patterns inherently present in temporal information.
 - **Rich Item Initialization:** Explore methods to generate semantically rich initial item embeddings, moving beyond standard random initialization. Incorporating auxiliary information or pre-trained representations based on inductive biases such as factors in [?], before the GNN operations, can establish a more meaningful initial adjacency matrix based on relations between item embeddings (e.g. representing this relations as edges), potentially enhancing the final representation of items in the latent space.
 - **Broader Evaluation Metrics:** Expand the evaluation protocol by incorporating a wider set of performance metrics (Such as the Bayesian Personalized Ranking and TOP1). Assessing different qualitative and quantitative aspects of the recommendations will yield a more multi-perspective understanding of the model's overall effectiveness and performance.

REFERENCES

- [1] Park, S. E., Lee, S., Lee, S.-g. (2011). *Session-based collaborative filtering for predicting the next song*. 2011 First ACIS/JNU International Conference on Computers, Networks, Systems and Industrial Engineering, 353-358. [<https://doi.org/10.1109/CNSI.2011.72>](<https://www.google.com/search?q=https://doi.org/10.1109/CNSI.2011.72>)
- [2] Su, Xiaoyuan, Khoshgoftaar, Taghi M., A Survey of Collaborative Filtering Techniques, *Advances in Artificial Intelligence*, 2009, 421425, 19 pages, 2009. <https://doi.org/10.1155/2009/421425>
- [3] Ruslan Salakhutdinov and Andriy Mnih. (2008). Bayesian probabilistic matrix factorization using markov chain Monte Carlo. In *Proceedings of the 25th International Conference on Machine Learning*
- [4] Bobadilla, J., Ortega, F., Hernando, A., Bernal, J. (2012). A collaborative filtering approach to mitigate the new user cold start problem. *Knowledge-Based Systems*, 26, 225-238. <https://doi.org/10.1016/j.knosys.2011.07.021>
- [5] Francisco J. Peña, Diarmuid O'Reilly-Morgan, Elias Z. Tragos, Neil Hurley, Erika Duriakova, Barry Smyth, and Aonghus Lawlor. (2020). Combining Rating and Review Data by Initializing Latent Factor Models with Topic Models for Top-N Recommendation. In *RecSys 2020 - 14th ACM Conference on Recommender Systems*.
- [6] Koren, Yehuda, Bell, Robert, and Volinsky, Chris. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37.
- [7] Weimer, Markus, Karatzoglou, Alexandros, Le, Quoc Viet, and Smola, Alex. (2007). Maximum margin matrix factorization for collaborative ranking. *Advances in neural information processing systems*.
- [8] Hidasi, B. and Tikk, D. (2012). Fast ALS-based tensor factorization for context-aware recommendation from implicit feedback. In *ECML-PKDD'12, Part II*, number 7524 in LNCS, pp. 67–82. Springer.

REFERENCES

- [9] Sarwar, Badrul, Karypis, George, Konstan, Joseph, and Riedl, John. (2001). Item-based collaborative filtering recommendation algorithms. In Proceedings of the 10th international conference on World Wide Web, pp. 285–295. ACM.
- [10] Koren, Y. (2008). Factorization meets the neighborhood: a multifaceted collaborative filtering model. In SIGKDD’08: ACM Int. Conf. on Knowledge Discovery and Data Mining, pp. 426–434.
- [11] Shani, G.; Brafman, R. I.; and Heckerman, D. (2002). An mdp-based recommender system. In UAI, 453–460.
- [12] Rendle, S.; Freudenthaler, C.; and Schmidt-Thieme, L. (2010). Factorizing personalized markov chains for next-basket recommendation. In WWW, 811–820. ACM.
- [13] Garg, D., Gupta, P., Malhotra, P., Vig, L., Shroff, G.: Sequence and Time Aware Neighborhood for Session-based Recommendations: STAN. In: Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 1069–1072. ACM, New York (2019). 10.1145/3331184.3331322
- [14] Tan, Y. K.; Xu, X.; and Liu, Y. (2016). Improved recurrent neural networks for session-based recommendations. In DLRS, 17–22.
- [15] Tuan, T. X., and Phuong, T. M. (2017). 3d convolutional networks for session-based recommendation with content features. In RecSys, 138–146.
- [16] Li, J.; Ren, P.; Chen, Z.; Ren, Z.; Lian, T.; and Ma, J. (2017). Neural attentive session-based recommendation. In CIKM, 1419–1428.
- [17] Shoujin Wang and Liang Hu and Yan Wang and Xiangnan He and Quan Z. Sheng and Mehmet A. Orgun and Longbing Cao and Francesco Ricci and Philip S. Yu, (2021), Graph Learning based Recommender Systems: A Review, <https://doi.org/10.48550/arXiv.2105.06339>
- [18] Sanchez-Lengeling, et al., "A Gentle Introduction to Graph Neural Networks", Distill, 2021.
- [19] Santhosh Rajamanickam, (2021), Graph Neural Network (GNN) Architectures for Recommendation Systems, <https://towardsdatascience.com/graph-neural-network-gnn-architectures-for-recommendation-systems-7b9dd0de0856/?gi=f357864deb6a>
- [20] Hidasi, B., Karatzoglou, A., Baltrunas, L., Tikk, D. (2015). *Session-based recommendations with recurrent neural networks.*, arXiv preprint, <https://doi.org/10.48550/arXiv.1511.06939>

-
- [21] Tajuddeen Rabiou Gwadabe and Ying Liu, *Improving graph neural network for session-based recommendation system via non-sequential interactions*, Neurocomputing, 2022. <https://doi.org/10.1016/j.neucom.2021.10.034>
- [22] Chhotelal Kumar, Mukesh Kumar, (2024), *Session-based recommendations with sequential context using attention-driven LSTM*, Computers and Electrical Engineering, Volume 115, 109138, ISSN 0045-7906, <https://doi.org/10.1016/j.compeleceng.2024.109138>.
- [23] Ehsan Elahi, Sajid Anwar, Mousa Al-kfairy, Joel J.P.C. Rodrigues, Alladoubaye Nguetilbaye, Zahid Halim, Muhammad Waqas, (2025), *Graph attention-based neural collaborative filtering for item-specific recommendation system using knowledge graph*, Expert Systems with Applications, Volume 266, 126133, ISSN 0957-4174, <https://www.sciencedirect.com/science/article/pii/S0957417424030008>.
- [24] Ji, W., Wang, K., Wang, X., Chen, T., Cristea, A. I. (2020). *Hybrid sequential recommender via time-aware attentive memory network*. arXiv preprint, [<https://doi.org/10.48550/arXiv.2005.08598>](<https://www.google.com/search?q=https://doi.org/10.48550/arXiv.2005.08598>)
- [25] Li, Y., Tarlow, D., Brockschmidt, M., Zemel, R. (2015). *Gated graph sequence neural networks*. arXiv preprint, <https://doi.org/10.48550/arXiv.1511.05493>
- [26] Wu, Shu and Tang, Yuyuan and Zhu, Yanqiao and Wang, Liang and Xie, Xing and Tan, Tieniu, (2018), *Session-Based Recommendation with Graph Neural Networks*, Proceedings of the AAAI Conference on Artificial Intelligence, Volume 33, ISSN 2159-5399, <http://dx.doi.org/10.1609/aaai.v33i01.3301346>
- [27] William L. Hamilton. (2020). Graph Representation Learning. Synthesis Lectures on Artificial Intelligence and Machine Learning, Vol. 14, No. 3 , Pages 1-159.
- [28] R. Salakhutdinov, A. Mnih, (2007). *Probabilistic matrix factorization*, Advances in Neural Information Processing Systems, Volume 20, ISN 1257-1264, https://proceedings.neurips.cc/paper_files/paper/2007/file/d7322ed717dedf1eb4e6e52a37ea7bcd-Paper.pdf
- [29] Y. Koren, R. Bell and C. Volinsky, (2009). *Matrix Factorization Techniques for Recommender Systems*, Computer, vol. 42, no. 8, pp. 30-37, 10.1109/MC.2009.263
- [30] Zhang, Liang Agarwal, Deepak Chen, Bee-Chung. (2011). Generalizing matrix factorization through flexible regression priors. RecSys'11 - Proceedings of the 5th ACM Conference on Recommender Systems. 13-20. 10.1145/2043932.2043940.
- [31] Shaina Raza and Mizanur Rahman and Safiullah Kamawal and Armin Toroghi and Ananya Raval and Farshad Navah and Amirmohammad Kazemeini. (2025). *A Comprehensive Review of Recommender Systems: Transitioning from Theory to Practice*, arXiv, arXiv:2407.13699

REFERENCES

- [32] W Ji, K. Wang, X. Wang, T. Chen, A. Cristea, (2017). Sequential Recommender via Time-aware Attentive Memory Network, ArXiv, <https://arxiv.org/abs/2005.08598>.
- [33] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., Polosukhin, I. (2017). Attention is all you need. arXiv. <https://doi.org/10.48550/arXiv.1706.03762>
- [34] P. Velićković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio. Graph attention networks. (2018), ICLR.
- [35] Hamilton, W. L., Ying, R., Leskovec, J. (2017). *Inductive representation learning on large graphs*. arXiv preprint, <https://doi.org/10.48550/arXiv.1706.02216>
- [36] Dey, Rahul, Salem, Fathi M., Gate-Variants of Gated Recurrent Unit (GRU) Neural Networks, CoRR, abs/1701.05923, 11 pages, 2017. <http://arxiv.org/abs/1701.05923>
- [37] Schmidt, Robin M., Recurrent Neural Networks (RNNs): A gentle Introduction and Overview, CoRR, abs/1912.05911, 2019. <http://arxiv.org/abs/1912.05911>